```
var _diagMsgs = [];
var _diagStart = Date.now();
var _diagModalShowing = false;
var _diagModalShown = false;
function _diag(msg) {
 var line = '[' + (Date.now() - _diagStart) + 'ms] ' + msg;
 _diagMsgs.push(line);
 try { console.log(line); } catch (_) {}
}
function _showDiag() {
 try {
  if (_diagModalShowing || _diagModalShown) {
   return;
  }
  if (typeof wx !== 'undefined' && wx.showModal) {
   _diagModalShowing = true;
   _diagModalShown = true;
   wx.showModal({
    title: '启动诊断',
    content: _diagMsgs.join('\n'),
    showCancel: false,
    fail: function(err) {
     try { console.warn('[diag] showModal failed', err); } catch (_) {}
    },
    complete: function() {
     _diagModalShowing = false;
    }
   });
  }
 } catch (_) {}
}
_diag('game.js start');
try {
 if (typeof wx !== 'undefined' && typeof wx.createCanvas === 'function' && typeof GameGlobal !==...
  // 借鉴 xiao_chu 的好友榜方案：第一个 createCanvas 作为真正上屏 Canvas，并锁定 2D context。
  // 微信开放数据域 sharedCanvas 只能 drawImage 到上屏 Canvas，不能进 WebGL/普通离屏 Canvas。
  // pixi-adapter 会在后面再创建一个 canvas 给 Pixi/WebGL 使用，Game.ts 每帧把 Pixi 离屏画面
  // 与 sharedCanvas 合成到这个 2D 上屏 Canvas。
  var _mainCanvas = wx.createCanvas();
  var _mainCtx = _mainCanvas.getContext('2d');
  var _screenInfo = wx.getSystemInfoSync ? wx.getSystemInfoSync() : null;
  var _dpr = (_screenInfo && _screenInfo.pixelRatio) || 2;
  var _screenW = (_screenInfo && (_screenInfo.windowWidth || _screenInfo.screenWidth)) || 375;
  var _screenH = (_screenInfo && (_screenInfo.windowHeight || _screenInfo.screenHeight)) || 667;
  _mainCanvas.width = _screenW * _dpr;
  _mainCanvas.height = _screenH * _dpr;
  if (_mainCtx) {
   _mainCtx.fillStyle = '#f7e4c4';
   _mainCtx.fillRect(0, 0, _mainCanvas.width, _mainCanvas.height);
  }
```

```
    GameGlobal.__mainCanvas = _mainCanvas;
    _diag('main 2d canvas ready:' + _mainCanvas.width + 'x' + _mainCanvas.height);
   }
} catch (e) {
  _diag('main 2d canvas failed:' + e);
}
try {
  if (typeof wx !== 'undefined' && wx.getSystemInfoSync) {
    var _info = wx.getSystemInfoSync();
    _diag('platform:' + _info.platform + ' system:' + _info.system);
    _diag('brand:' + _info.brand + ' model:' + _info.model);
    _diag('SDK:' + _info.SDKVersion);
  }
} catch (e) {
  _diag('systemInfo failed:' + e);
}
try {
  if (typeof GameGlobal !== 'undefined') {
    GameGlobal.onError = function(msg) {
      _diag('onError:' + msg);
      _showDiag();
    };
    GameGlobal.onUnhandledRejection = function(ev) {
      _diag('unhandledRejection:' + (ev && ev.reason || ev));
      _showDiag();
    };
  }
} catch (_) {}
try {
  require('./minigame/pixi-adapter/index');
  _diag('pixi-adapter ok');
} catch (e) {
  _diag('pixi-adapter failed:' + e);
  _showDiag();
}
// 部分安卓/鸿蒙微信小游戏 JS 引擎没有 Intl，PixiJS 文本测量里直接访问 Intl 会导致启动阶段 ReferenceError。
if (typeof Intl === 'undefined') {
  _diag('Intl missing, install stub');
  try {
    if (typeof GameGlobal !== 'undefined') GameGlobal.Intl = {};
    if (typeof globalThis !== 'undefined') globalThis.Intl = {};
  } catch (_) {}
}
try {
  require('./minigame/game-bundle.js');
  _diag('game-bundle ok');
} catch (e) {
  _diag('game-bundle failed:' + e);
  _showDiag();
}
```

```
try {
  setTimeout(function() {
    if (typeof GameGlobal !== 'undefined' && !GameGlobal.__hotPotRendered) {
      _diag('5s timeout: game not rendered');
      _showDiag();
    }
  }, 5000);
} catch (_) {}
import { Analytics, EVENT_NAMES, type DeviceInfo, type PlatformName } from '@gp/analytics-sdk';
import { BACKEND_BASE_URL } from '@/config/CloudConfig';
import { Platform } from '@/core/PlatformService';
export { EVENT_NAMES };
export const analytics = Analytics;
const APP_VERSION = '1.0.0';
const GAME_KEY = 'hotpot';
/**
 * 复用 hot-pot 现有 hotpot-api 同款 HTTP 访问服务网关:
 *   <env-id>.service.tcloudbase.com/<云函数名>/<path>
 * 这是 CloudBase「HTTP 访问服务」网关，跟「云函数 HTTP 触发器」是两套不同 URL 格式，
 * 之前写的 *.app.tcloudbase.com/... 是后者的格式，会被网关 404 INVALID_PATH 拒绝。
 */
const ENDPOINT = `${BACKEND_BASE_URL}/analytics-ingest/track`;
let inited = false;
/** hot-pot 接入入口：在 main.ts 启动尽早调用一次。把 PlatformService 作为 Adapter 注入给 SDK。 */
export function initAnalytics(opts?: { endpoint?: string; userId?: string; debug?: boolean }): vo...
  if (inited) return;
  const platformName = mapPlatform();
  const deviceInfo = buildDeviceInfo();
  Analytics.init({
    endpoint: opts?.endpoint || ENDPOINT,
    gameKey: GAME_KEY,
    appVersion: APP_VERSION,
    platform: platformName,
    deviceInfo,
    initialUserId: opts?.userId,
    transport: { request: Platform.request.bind(Platform) },
    storage: {
      get: Platform.getStorageSync.bind(Platform),
      set: Platform.setStorageSync.bind(Platform),
      remove: Platform.removeStorageSync.bind(Platform),
    },
    lifecycle: { onHide: Platform.onHide.bind(Platform) },
    debug: opts?.debug,
  });
  inited = true;
}
/** 业务登录拿到 openid 后调用，让后续事件都带上 user_id */
export function setAnalyticsUserId(userId: string): void {
  if (!inited) return;
  Analytics.setUserId(userId || '');
```

```
}
function mapPlatform(): PlatformName {
  if (Platform.isWechat) return 'wechat';
  if (Platform.isDouyin) return 'douyin';
  if (Platform.isMinigame) return 'unknown';
  return 'h5';
}
function buildDeviceInfo(): DeviceInfo {
  const sys = Platform.getSystemInfoSync() || {};
  return {
    brand: String(sys.brand || sys.deviceBrand || ''),
    model: String(sys.model || sys.deviceModel || ''),
    system: String(sys.system || ''),
    sdkVersion: String(sys.SDKVersion || sys.sdkVersion || ''),
    screenWidth: Number(sys.screenWidth) || 0,
    screenHeight: Number(sys.screenHeight) || 0,
    network: 'unknown',
  };
}
/** CloudBase HTTP backend configuration for the hotpot game. */
export const GAME_KEY = 'hotpot';
export const BACKEND_BASE_URL = 'https://rosa-env-d7grf78r5dbd37323.service.tcloudbase.com';
export const BACKEND_PATH_PREFIX = `/${GAME_KEY}-api`;
export const CLOUD_ENV_ID = 'rosa-env-d7grf78r5dbd37323';
export const BACKEND_LOGIN_PATH = `${BACKEND_PATH_PREFIX}/login/`;
export const BACKEND_PULL_PATH = `${BACKEND_PATH_PREFIX}/save/pull`;
export const BACKEND_PUSH_PATH = `${BACKEND_PATH_PREFIX}/save/push`;
export const BACKEND_HEALTH_PATH = `${BACKEND_PATH_PREFIX}/health`;
export const BACKEND_RANK_SUBMIT_PATH = `${BACKEND_PATH_PREFIX}/rank/submit`;
export const BACKEND_RANK_LIST_PATH = `${BACKEND_PATH_PREFIX}/rank/list`;
export const BACKEND_RANK_MINE_PATH = `${BACKEND_PATH_PREFIX}/rank/mine`;
export const BACKEND_LEVEL_PASS_RATES_PATH = `${BACKEND_PATH_PREFIX}/level/pass-rates`;
export const BACKEND_GAME_CLUB_DECRYPT_PATH = `${BACKEND_PATH_PREFIX}/game-club/decrypt`;
export const LEVEL_PASS_RATE_COLLECTION = 'hotpot_public_level_pass_rates';
export const LEVEL_PASS_RATE_DOC_ID = 'bowl_30d_latest';
export const BACKEND_REQUEST_TIMEOUT_MS = 10000;
export const BACKEND_TOKEN_KEY = `${GAME_KEY}_token`;
export const BACKEND_ANON_ID_KEY = `${GAME_KEY}_anon_id`;
export const BOWL_PROGRESS_KEY = `${GAME_KEY}_bowl_progress`;
export const USER_SETTINGS_KEY = `${GAME_KEY}_settings`;
export const FRUIT_SLICE_PROGRESS_KEY = `${GAME_KEY}_fruit_slice_progress`;
export const WALLET_KEY = `${GAME_KEY}_wallet_v1`;
export const GACHA_STATE_KEY = `${GAME_KEY}_gacha_state_v1`;
export const TOOL_INVENTORY_KEY = `${GAME_KEY}_tool_inventory_v1`;
export const FRUIT_SLICE_REWARD_KEY = `${GAME_KEY}_fruit_slice_reward_v1`;
export const FRUIT_SLICE_TOOL_INVENTORY_KEY = `${GAME_KEY}_fruit_slice_tool_inventory_v1`;
export const GAME_CLUB_REWARD_KEY = `${GAME_KEY}_game_club_reward_v1`;
export const LEVEL_PASS_RATE_CACHE_KEY = `${GAME_KEY}_level_pass_rates_v1`;
/**
 * 玩家自己授权拿到的微信昵称 + 头像 URL；仅用于排行榜显示。
```

```
 * 不进入 CLOUD_SYNC_ALLOWLIST：用户资料本机敏感，不参与跨设备云端同步，
 * 换设备时由玩家在新设备上重新点击授权即可。
 */
export const USER_PROFILE_KEY = `${GAME_KEY}_user_profile`;
export const CLOUD_SYNC_SCHEMA_VERSION = 1;
export const CLOUD_SYNC_META_KEY = `${GAME_KEY}_cloud_meta`;
export const CLOUD_SYNC_ALLOWLIST = [
  BOWL_PROGRESS_KEY,
  USER_SETTINGS_KEY,
  FRUIT_SLICE_PROGRESS_KEY,
  WALLET_KEY,
  GACHA_STATE_KEY,
  TOOL_INVENTORY_KEY,
  FRUIT_SLICE_REWARD_KEY,
  FRUIT_SLICE_TOOL_INVENTORY_KEY,
  GAME_CLUB_REWARD_KEY,
] as const;
export const CLOUD_SYNC_EXCLUDE_KEYS = [
  BACKEND_TOKEN_KEY,
  BACKEND_ANON_ID_KEY,
] as const;
export const CLOUD_SYNC_STARTUP_TIMEOUT_MS = 2500;
export const CLOUD_SYNC_DEBOUNCE_MS = 1500;
export const CLOUD_SYNC_BASE_DELAY_MS = 1500;
export const CLOUD_SYNC_MAX_BACKOFF_MS = 30000;
export const CLOUD_SYNC_MAX_FAIL_COUNT = 5;
export const CLOUD_SYNC_RETRY_INTERVAL_MS = 60000;
export const CLOUD_SYNC_LOG_THRESHOLD = 3;
export type CloudSyncKey = typeof CLOUD_SYNC_ALLOWLIST[number];
/**
 * 玩法相关贴图放在微信分包，降低主包体积（主包仅保留首页等）。
 * 路径相对小游戏项目根目录（与 game.json 中 subpackages.bowl_game 一致）。
 */
export const BOWL_IMAGES_ROOT = 'subpackages/bowl_game/assets/images';
export const BOWL_CORE_IMAGES_ROOT = 'subpackages/bowl_core/assets/images';
export const BOWL_THEME_IMAGES_ROOT = 'subpackages/bowl_themes/assets/images/themes';
export const BOWL_BADGE_IMAGES_ROOT = 'subpackages/bowl_badges/assets/images/badges';
export const CATALOG_IMAGES_ROOT = 'subpackages/catalog_assets/assets/images';
export const DAILY_LIMITED_IMAGES_ROOT = 'subpackages/daily_limited/assets/images/daily_limited';
export const DAILY_RECIPE_IMAGES_ROOT = 'subpackages/daily_recipes/assets/images/daily_limited/re...
export const FRUIT_SLICE_IMAGES_ROOT = 'subpackages/fruit_slice/assets/images/fruit_slice';
export const GACHA_IMAGES_ROOT = 'subpackages/gacha_assets/assets/images/gacha';
import { BOWL_BADGE_IMAGES_ROOT } from '@/config/bowlAssets';
export interface BowlBadgeDef {
  levelNumber: number;
  title: string;
  asset: string;
  vessel: 'cup' | 'bowl';
  drinkColor: number;
  accentColor: number;
```

```
  fruitColors: readonly number[];
  garnishColor: number;
}
const badge = (
  levelNumber: number,
  title: string,
  vessel: BowlBadgeDef['vessel'],
  drinkColor: number,
  accentColor: number,
  fruitColors: readonly number[],
  garnishColor: number,
): BowlBadgeDef => ({
  levelNumber,
  title,
  asset: `${BOWL_BADGE_IMAGES_ROOT}/bowl_badge_${String(levelNumber).padStart(2, '0')}.png`,
  vessel,
  drinkColor,
  accentColor,
  fruitColors,
  garnishColor,
});
export const BOWL_BADGES: BowlBadgeDef[] = [
  badge(1, '酸奶莓果杯', 'cup', 0xfff4e7, 0x7ed8ff, [0xff5a7a, 0x355fd1, 0xffc94d], 0x65c85a),
  badge(2, '柠檬蓝莓冰茶', 'cup', 0xffe66d, 0x68d6ff, [0xffe34d, 0x3c73df, 0xff7aaa], 0x50bc6b),
  badge(3, '热带芒芒刨冰', 'bowl', 0xffb43d, 0xffec88, [0xff9f2f, 0xff4c61, 0xffdd4c], 0x62c765),
  badge(4, '星星西瓜水果捞', 'bowl', 0xff6b82, 0x93f0ff, [0xff3f63, 0x3ed56b, 0xffe95c], 0x48b65c),
  badge(5, '莓柚气泡冰杯', 'cup', 0xff7aa8, 0xb8efff, [0xff4b7d, 0xbb3fe3, 0xffc34f], 0x71cf64),
  badge(6, '坚果蜜桃雪顶杯', 'cup', 0xffc17f, 0xfff0b3, [0xffa46b, 0xd8a05a, 0xffe087], 0x78bd52),
  badge(7, '椰香桂圆冰碗', 'bowl', 0xffead0, 0x8fe7ff, [0xffffff, 0xe7a858, 0xffd967], 0x7ac568),
  badge(8, '多彩小料奶茶', 'cup', 0xd9a06a, 0x83ddff, [0xff7acd, 0xffd54a, 0x6c55d8], 0x58bd70),
  badge(9, '脆脆莓莓冰沙', 'bowl', 0xf55e92, 0xbdf4ff, [0xff5280, 0xffd447, 0x8e5ee7], 0x75c85d),
  badge(10, '双轨橙柚冰茶', 'cup', 0xff9d35, 0x68d8ff, [0xffd24a, 0xff6b42, 0x7bd85a], 0x5bc46d),
  badge(11, '十二鲜果冰碗', 'bowl', 0xffefb4, 0x9befff, [0xff515f, 0x3c7ee8, 0xffdc54, 0x70d95d], 0x60b...
  badge(12, '续章荔枝水果茶', 'cup', 0xffd9e8, 0x96e8ff, [0xff78a2, 0xfff2f2, 0xffc14f], 0x6ac66a),
  badge(13, '东方梅子冰饮', 'cup', 0xb95fc2, 0xffd87b, [0x8a3f9e, 0xff6b86, 0xffe066], 0x64bd70),
  badge(14, '混搭雪梨刨冰', 'bowl', 0xf7f5dc, 0x89ddff, [0xffd552, 0xff7a4d, 0x7edc70], 0x5ec06c),
  badge(15, '蜜瓜桃桃冰杯', 'cup', 0xbef17a, 0xffb7d0, [0xff9a60, 0xc8f36a, 0xffe45a], 0x5abf65),
  badge(16, '满贯浆果雪山', 'bowl', 0xd46be8, 0xb5efff, [0xff507b, 0x4f6ce8, 0xffd557], 0x73ca62),
  badge(17, '满贯椰椰冰茶', 'cup', 0xfff4d6, 0x7ee0ff, [0xffffff, 0xffcc58, 0x7bd961], 0x55bd67),
  badge(18, '杂烩水果捞碗', 'bowl', 0xffc25f, 0x9defff, [0xff5b61, 0x7f56df, 0xffe156, 0x5fce65], 0x58b...
  badge(19, '薄冰青柠杯', 'cup', 0xcff377, 0xa9f3ff, [0xc6f04e, 0xffffff, 0x68d8ff], 0x51b96a),
  badge(20, '四合莲藕冰碗', 'bowl', 0xf2dfc9, 0x88ddff, [0xf2c28a, 0xffe06a, 0xb87bdc], 0x6bc15f),
  badge(21, '果阵红莓冰沙', 'cup', 0xff5d73, 0xffd1e0, [0xff3f5d, 0x405bdc, 0xffd54d], 0x68c767),
  badge(22, '果阵青柠雪碗', 'bowl', 0xaee95e, 0x8ee7ff, [0xb9ef4e, 0xffdd4a, 0x68d9ff], 0x56bf68),
  badge(23, '滋补银耳甜品', 'bowl', 0xffefd5, 0xf6c17d, [0xffffff, 0xc06ad9, 0xffd760], 0x7ac160),
  badge(24, '小料珍珠奶茶', 'cup', 0xcf9863, 0x83dbff, [0x5a3428, 0xff7fc8, 0xffdc56], 0x62bd66),
  badge(25, '重口巧克力冰杯', 'cup', 0x8b5a38, 0xffc968, [0x5b3524, 0xfff2e0, 0xff6a84], 0x68bd59),
  badge(26, '果下芒果椰雪', 'bowl', 0xffc53f, 0xffef91, [0xffa438, 0xffffff, 0xffdd4a], 0x65c45d),
  badge(27, '滋补红枣冰饮', 'cup', 0xb84f45, 0xffd18a, [0xa64038, 0xffda65, 0xf4eee0], 0x6cc062),
  badge(28, '小料仙草冰碗', 'bowl', 0x493d45, 0x8bdcff, [0x302934, 0xffc84a, 0xff78c7], 0x65bd67),
```

```
  badge(29, '甘星爆珠水果茶', 'cup', 0xff8b47, 0x92e9ff, [0xff3f6d, 0xffdb4a, 0x5f70e8, 0x61d35b], 0x5ac...
  badge(30, '终章龙果冰冠', 'bowl', 0xff56aa, 0xffe06a, [0xff2f91, 0xffffff, 0xffd84e, 0x63d966], 0x59c...
  badge(31, '紫冠雪梨杯', 'cup', 0xf4e8ff, 0xffe58a, [0x6a1b9a, 0xf6e7a8, 0xffffff], 0x78c968),
  badge(32, '牛油果奶昔杯', 'cup', 0xc8f0a8, 0xfff4d6, [0x7fba3f, 0xf5efcf, 0x5c7a2f], 0x66bd5f),
  badge(33, '红宝石石榴冰', 'bowl', 0xd9324c, 0xffd78a, [0xc91f3c, 0xff6f86, 0xfff1d6], 0x5fc36b),
  badge(34, '弯月山楂冰茶', 'cup', 0xffd77a, 0xff8b7a, [0xbce874, 0xd73338, 0xfff0a8], 0x63bf63),
  badge(35, '莲雾粉雾杯', 'cup', 0xffb8c8, 0xa8efff, [0xff5f86, 0xffd6e0, 0xffffff], 0x72c76a),
  badge(36, '油柑回甘冰饮', 'cup', 0xbde86b, 0xffdf72, [0x90c84a, 0xdff59a, 0xfff0b8], 0x5ebd68),
  badge(37, '芭乐餐车奶昔', 'cup', 0xd9f0a8, 0xffcfa8, [0x9fd66a, 0xffb6b6, 0xffffff], 0x62bf68),
  badge(38, '木瓜暖奶碗', 'bowl', 0xffb15a, 0xffefd1, [0xff8f2e, 0xffcf67, 0xffffff], 0x67c35d),
  badge(39, '无花果蜜宴碗', 'bowl', 0xb85a93, 0xffc76f, [0x7d3a8c, 0xd96f9f, 0xffe08a], 0x6bc064),
  badge(40, '杏光沙滩终宴', 'bowl', 0xffb55a, 0xffe077, [0xff9c3d, 0xffd06a, 0xfff2c0, 0x63d966], 0x58c...
];
export function getBowlBadgeDef(levelNumber: number): BowlBadgeDef {
  const clamped = Math.max(1, Math.min(levelNumber, BOWL_BADGES.length));
  return BOWL_BADGES[clamped - 1]!;
}
import type { FruitId } from '@/config/fruits';
import type { BowlRimKey, BowlSoupKey } from '@/config/bowlSkins';
import type { BowlThemeKey } from '@/config/bowlThemes';
export interface BowlLevelDef {
  /** 关卡序号（从 1 起，与 UI「第 N 关」一致） */
  levelNumber: number;
  displayName: string;
  /** 本关出现的可下单食材种类 */
  fruitIds: FruitId[];
  /** 每种可下单食材在碗内出现的个数 */
  copiesPerFruit: number;
  /** 每个订单需要的个数 */
  orderTarget: number;
  /** 底部暂存槽位数 */
  bufferSize: number;
  /** 冰块障碍件数：不进订单，但会占碗与暂存压力 */
  iceCount?: number;
  /**
   * 冻果件数：从本关订单库存中挑 N 颗盖上冰块。
   * 点击会强制进暂存槽并开始解冻；解冻后按普通水果交付，仍计入订单总数。
   */
  frozenCount?: number;
  /** 开局浮在上层、可点击的物品上限；其余先藏在下层，随订单推进浮出 */
  initialVisibleCount?: number;
  /** 每完成一盘订单后，从下层释放的物品数量 */
  revealPerOrderComplete?: number;
  /** 开局并行订单路数，默认 2；新手加速关可直接开到 4 路 */
  plateLanesInitial?: 2 | 3 | 4;
  allowAddDish: boolean;
  allowRemove: boolean;
  allowShuffle: boolean;
  soupKey?: BowlSoupKey;
  bowlKey?: BowlRimKey;
```

```
  /** 玩法页主题；未配置时按关卡区间自动切换 */
  themeKey?: BowlThemeKey;
}
const allTools = { allowAddDish: true, allowRemove: true, allowShuffle: true } as const;
/**
 * 每关新解锁的食材（累积式：本关订单池 = 前 N 组并集）。
 * 节奏：L1=4、L2 解锁 4 但订单池压到 7、L3 解锁 4 但订单池压到 11；
 *     L4-15 保持原解锁节奏不动，L16-30 补齐全部小料/滋补/甜品食材，
 *     L31-40 开启鲜果续章。
 * 关卡通关弹窗的「下一关解锁」会显示对应组中的新水果。
 */
const UNLOCK_GROUPS = [
  ['blueberry', 'lemon', 'orange', 'strawberry'],
  ['apple', 'banana', 'grape', 'kiwi'],
  ['cucumber', 'peach', 'pineapple', 'watermelon'],
  ['mango', 'cherry', 'cherry_tomato'],
  ['grape_green', 'lime', 'mandarin'],
  ['cantaloupe', 'honeydew', 'young_coconut'],
  ['lychee', 'longan'],
  ['dried_longan', 'bayberry'],
  ['blackberry', 'cranberry'],
  ['raspberry', 'mulberry'],
  ['passionfruit', 'grapefruit'],
  ['kumquat', 'starfruit'],
  ['plum', 'nectarine'],
  ['persimmon', 'almond_slice'],
  ['peanut', 'walnut_piece'],
  ['chestnut', 'oat_flake'],
  ['red_date', 'sour_plum', 'mint'],
  ['blackcurrant', 'gooseberry', 'osmanthus'],
  ['basil_seed', 'crystal_jelly'],
  ['lotus_root', 'water_chestnut', 'radish_heart'],
  ['black_rice', 'red_bean'],
  ['foxnut', 'lotus_seed'],
  ['lily_bulb', 'snow_fungus', 'peach_gum'],
  ['boba_pearl', 'pudding_cube', 'mini_mochi'],
  ['chocolate_chip', 'cookie_crumb', 'marshmallow'],
  ['pumpkin_cube', 'sweet_potato', 'durian'],
  ['coconut_jelly', 'sago'],
  ['grass_jelly', 'taro_ball', 'taro_dice'],
  ['pop_boba'],
  ['dragonfruit'],
  ['mangosteen', 'pear'],
  ['avocado'],
  ['pomegranate'],
  ['yangjiaomi_melon', 'hawthorn'],
  ['wax_apple'],
  ['emblic'],
  ['guava'],
  ['papaya'],
```

```
  ['fig'],
  ['apricot'],
] as const satisfies readonly (readonly FruitId[])[];
/** 图鉴只展示关卡体系内真正可获得的食材，避免通关后仍出现永远无法解锁的占位。 */
export const BOWL_UNLOCKABLE_FRUIT_IDS = Array.from(new Set<FruitId>(UNLOCK_GROUPS.flat()));
/**
 * 后 15 关不再把最早期全部基础水果无限累积进订单池，而是保留一批辨识度高的主食材，
 * 再叠加最近解锁的小料/滋补/甜品，保证每关新鲜感和图鉴解锁一致。
 */
const POST_15_BASE_POOL = [
  'apple',
  'banana',
  'grape',
  'kiwi',
  'peach',
  'pineapple',
  'watermelon',
  'mango',
  'cherry',
  'cherry_tomato',
  'grape_green',
  'lime',
  'mandarin',
  'cantaloupe',
  'honeydew',
  'young_coconut',
  'lychee',
  'longan',
  'bayberry',
  'blackberry',
  'cranberry',
  'raspberry',
  'mulberry',
  'passionfruit',
  'grapefruit',
  'plum',
  'nectarine',
  'persimmon',
  'almond_slice',
  'peanut',
  'walnut_piece',
] as const satisfies readonly FruitId[];
const POST_15_RECENT_GROUP_WINDOW = 10;
const POST_15_FIRST_LEVEL = 16;
const POST_15_START_ORDER_COUNT = 54;
const POST_15_ORDER_INCREMENT = 1;
const POST_15_COPIES_PER_FRUIT = 6;
const FINAL_CHAPTER_FIRST_LEVEL = 31;
const FINAL_CHAPTER_START_ORDER_COUNT = 72;
const FINAL_CHAPTER_END_ORDER_COUNT = 100;
```

```
function uniqueFruitIds(ids: readonly FruitId[]): FruitId[] {
  return Array.from(new Set<FruitId>(ids));
}
function post15TargetFoodCount(levelNumber: number): number {
  const orderCount = post15TargetOrderCount(levelNumber);
  return Math.ceil((orderCount * 3) / POST_15_COPIES_PER_FRUIT);
}
function post15TargetOrderCount(levelNumber: number): number {
  if (levelNumber >= FINAL_CHAPTER_FIRST_LEVEL) {
    const progress = Math.max(0, Math.min(1, (levelNumber - FINAL_CHAPTER_FIRST_LEVEL) / 9));
    return Math.round(
      FINAL_CHAPTER_START_ORDER_COUNT +
        (FINAL_CHAPTER_END_ORDER_COUNT - FINAL_CHAPTER_START_ORDER_COUNT) * progress,
    );
  }
  return POST_15_START_ORDER_COUNT + (levelNumber - POST_15_FIRST_LEVEL) * POST_15_ORDER_INCREMENT;
}
function targetFruitCountForEarlyLevel(levelNumber: number): number {
  const targets = [4, 7, 10, 12, 14, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25];
  return targets[Math.max(0, Math.min(levelNumber - 1, targets.length - 1))]!;
}
function trimOldFruitsForEarlyLevel(levelIndex: number, fruits: FruitId[]): FruitId[] {
  const target = targetFruitCountForEarlyLevel(levelIndex + 1);
  if (fruits.length <= target) {
    return fruits;
  }
  /**
   * 降低每关水果种类时，保护本关新解锁食材；L3 起额外保护上一关食材。
   * 其余名额从较新的旧食材往回补，优先剔除更早的旧水果。
   */
  const protectedStart = levelIndex <= 1 ? levelIndex : levelIndex - 1;
  const protectedIds = new Set<FruitId>(uniqueFruitIds(UNLOCK_GROUPS.slice(protectedStart, levelI...
  const keep = new Set<FruitId>(protectedIds);
  for (const id of fruits.filter((fruitId) => !protectedIds.has(fruitId)).reverse()) {
    if (keep.size >= target) {
      break;
    }
    keep.add(id);
  }
  return fruits.filter((id) => keep.has(id));
}
/**
 * 累积式：第 N 关订单池 = UNLOCK_GROUPS[0..N-1] 全部并集（去重）。
 * L16 起改为「后期基础池 + 最近新解锁食材」，避免早期食材过度重复。
 */
function levelFruits(levelNumber: number): FruitId[] {
  const idx = Math.max(0, Math.min(levelNumber - 1, UNLOCK_GROUPS.length - 1));
  if (idx < 15) {
    return trimOldFruitsForEarlyLevel(
      idx,
```

```
    uniqueFruitIds(UNLOCK_GROUPS.slice(0, idx + 1).flat()),
   );
  }
  const recentStart = Math.max(15, idx - POST_15_RECENT_GROUP_WINDOW + 1);
  const targetFoodCount = Math.max(
   1,
   levelNumber >= FINAL_CHAPTER_FIRST_LEVEL
    ? post15TargetFoodCount(levelNumber)
    : post15TargetFoodCount(levelNumber) - 1,
  );
  return uniqueFruitIds([
   ...UNLOCK_GROUPS.slice(recentStart, idx + 1).flat(),
   ...POST_15_BASE_POOL,
   ...UNLOCK_GROUPS.slice(0, recentStart).flat(),
  ]).slice(0, targetFoodCount);
}
/**
 * 40 关数值（v8，平缓学习曲线 + 碗内"满当当"视觉）：
 *  `orderTarget` 全程 = 3（三消核心玩法不动）；
 *  `copiesPerFruit` 必须全程为 3 的倍数，避免生成无法凑满 x3 的尾数水果。
 *  `plateLanesInitial` 默认 = 2 -- 第 3/4 路订单盘上的「解锁」按钮点了才看广告解锁
 *    L2 例外：首个扩展教学关直接开 4 路，让玩家体验多订单更快过关；
 *    （`unlockNextOrderPlateReward`）；失败复活也会顺带解锁一路。这是核心广告变现位。
 *  底部「加菜碟」工具是另一条广告点：每次 +1 个 `bufferSize`（最多 7），
 *    缓解 buffer 满压力但不解锁订单路。
 *  难度递增完全靠：水果种类 ^、copiesPerFruit、冰 / 冻果 ^、bufferSize v、初始可见 ^。
 *
 *  A 段 教学（L1-2）          - 零障碍上手；L2 起 7 种水果让选择压力出现
 *  B 段 习惯养成（L3-7）       - L3 起只增加水果种类与少量冰块，不再把订单量翻倍
 *  C 段 中阶（L8-15）          - 逐步加入冻果，障碍缓慢爬升，始终保留 5 个暂存盘
 *  D 段 长线（L16-30）         - 订单量小幅增长，靠新食材与背景变化提供新鲜感
 *  E 段 续章（L31-40）         - 新鲜果回归，叠加新碗汤奖励，不再追加复杂机制
 *
 * 总订单单位 (`ordersRemaining`)：L1=4 -> L3≈20 -> L10≈40 -> L15≈50 -> L16≈54 -> L30≈68 -> L40≈100
 * `bufferSize`：全程 5 格保留安全感；加菜碟工具仍可扩到 7 格。
 * `initialVisibleCount`：跟随订单量慢慢提升；通过水果放大和更多上层水果来保持画面丰富，
 *  不再用过量可点击水果制造早期压迫感。
 * `allTools` 全程开。
 */
export const BOWL_LEVELS: BowlLevelDef[] = [
  {
   levelNumber: 1,
   displayName: '第1关 酸奶初醒',
   fruitIds: levelFruits(1),
   copiesPerFruit: 3,
   orderTarget: 3,
   bufferSize: 5,
   initialVisibleCount: 14,
   revealPerOrderComplete: 4,
   plateLanesInitial: 2,
```

```
    ...allTools,
  },
  {
    levelNumber: 2,
    displayName: '第2关 鲜果开张',
    fruitIds: levelFruits(2),
    copiesPerFruit: 6,
    orderTarget: 3,
    bufferSize: 5,
    initialVisibleCount: 32,
    revealPerOrderComplete: 5,
    plateLanesInitial: 4,
    ...allTools,
  },
  {
    levelNumber: 3,
    displayName: '第3关 热带开席',
    fruitIds: levelFruits(3),
    copiesPerFruit: 6,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 2,
    initialVisibleCount: 54,
    revealPerOrderComplete: 5,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 4,
    displayName: '第4关 星果长廊',
    fruitIds: levelFruits(4),
    copiesPerFruit: 6,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 3,
    initialVisibleCount: 60,
    revealPerOrderComplete: 5,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 5,
    displayName: '第5关 薄冰试饮',
    fruitIds: levelFruits(5),
    copiesPerFruit: 6,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 4,
    frozenCount: 1,
    initialVisibleCount: 66,
```

```
    revealPerOrderComplete: 5,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 6,
    displayName: '第6关 坚果蜜语',
    fruitIds: levelFruits(6),
    copiesPerFruit: 6,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 4,
    frozenCount: 1,
    initialVisibleCount: 72,
    revealPerOrderComplete: 5,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 7,
    displayName: '第7关 暖盅小宴',
    fruitIds: levelFruits(7),
    copiesPerFruit: 6,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 5,
    frozenCount: 1,
    initialVisibleCount: 78,
    revealPerOrderComplete: 6,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 8,
    displayName: '第8关 满席小酌',
    fruitIds: levelFruits(8),
    copiesPerFruit: 6,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 5,
    frozenCount: 2,
    initialVisibleCount: 84,
    revealPerOrderComplete: 6,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 9,
    displayName: '第9关 甜脆交锋',
    fruitIds: levelFruits(9),
```

```
    copiesPerFruit: 6,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 6,
    frozenCount: 2,
    initialVisibleCount: 90,
    revealPerOrderComplete: 6,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 10,
    displayName: '第10关 深汤追单',
    fruitIds: levelFruits(10),
    copiesPerFruit: 6,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 6,
    frozenCount: 2,
    initialVisibleCount: 96,
    revealPerOrderComplete: 6,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 11,
    displayName: '第11关 鲜果巡礼',
    fruitIds: levelFruits(11),
    copiesPerFruit: 6,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 7,
    frozenCount: 2,
    initialVisibleCount: 100,
    revealPerOrderComplete: 6,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 12,
    displayName: '第12关 果香续宴',
    fruitIds: levelFruits(12),
    copiesPerFruit: 6,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 7,
    frozenCount: 3,
    initialVisibleCount: 104,
    revealPerOrderComplete: 6,
    plateLanesInitial: 2,
```

```
    ...allTools,
  },
  {
    levelNumber: 13,
    displayName: '第13关 东方蜜径',
    fruitIds: levelFruits(13),
    copiesPerFruit: 6,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 8,
    frozenCount: 3,
    initialVisibleCount: 108,
    revealPerOrderComplete: 7,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 14,
    displayName: '第14关 缤纷拼盘',
    fruitIds: levelFruits(14),
    copiesPerFruit: 6,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 8,
    frozenCount: 4,
    initialVisibleCount: 112,
    revealPerOrderComplete: 7,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 15,
    displayName: '第15关 百味协奏',
    fruitIds: levelFruits(15),
    copiesPerFruit: 6,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 9,
    frozenCount: 4,
    initialVisibleCount: 116,
    revealPerOrderComplete: 7,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 16,
    displayName: '第16关 十五星上席',
    fruitIds: levelFruits(16),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
```

```
    bufferSize: 5,
    iceCount: 9,
    frozenCount: 4,
    initialVisibleCount: 118,
    revealPerOrderComplete: 7,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 17,
    displayName: '第17关 十五星收束',
    fruitIds: levelFruits(17),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 9,
    frozenCount: 5,
    initialVisibleCount: 120,
    revealPerOrderComplete: 7,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 18,
    displayName: '第18关 杂味圆舞',
    fruitIds: levelFruits(18),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 10,
    frozenCount: 5,
    initialVisibleCount: 122,
    revealPerOrderComplete: 7,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 19,
    displayName: '第19关 薄冰试炼',
    fruitIds: levelFruits(19),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 10,
    frozenCount: 5,
    initialVisibleCount: 124,
    revealPerOrderComplete: 8,
    plateLanesInitial: 2,
    ...allTools,
  },
```

```
  {
    levelNumber: 20,
    displayName: '第20关 四味重奏',
    fruitIds: levelFruits(20),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 11,
    frozenCount: 6,
    initialVisibleCount: 126,
    revealPerOrderComplete: 8,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 21,
    displayName: '第21关 果阵初章',
    fruitIds: levelFruits(21),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 11,
    frozenCount: 6,
    initialVisibleCount: 128,
    revealPerOrderComplete: 8,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 22,
    displayName: '第22关 果阵回环',
    fruitIds: levelFruits(22),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 12,
    frozenCount: 6,
    initialVisibleCount: 130,
    revealPerOrderComplete: 8,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 23,
    displayName: '第23关 滋补雅集',
    fruitIds: levelFruits(23),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 12,
```

```
    frozenCount: 7,
    initialVisibleCount: 132,
    revealPerOrderComplete: 8,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 24,
    displayName: '第24关 小料满仓',
    fruitIds: levelFruits(24),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 13,
    frozenCount: 7,
    initialVisibleCount: 134,
    revealPerOrderComplete: 8,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 25,
    displayName: '第25关 重味温习',
    fruitIds: levelFruits(25),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 13,
    frozenCount: 7,
    initialVisibleCount: 136,
    revealPerOrderComplete: 9,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 26,
    displayName: '第26关 果阵再开',
    fruitIds: levelFruits(26),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 14,
    frozenCount: 8,
    initialVisibleCount: 138,
    revealPerOrderComplete: 9,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 27,
```

```
    displayName: '第27关 滋补再炖',
    fruitIds: levelFruits(27),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 14,
    frozenCount: 8,
    initialVisibleCount: 140,
    revealPerOrderComplete: 9,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 28,
    displayName: '第28关 小料再添',
    fruitIds: levelFruits(28),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 15,
    frozenCount: 8,
    initialVisibleCount: 142,
    revealPerOrderComplete: 9,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 29,
    displayName: '第29关 廿星连珠',
    fruitIds: levelFruits(29),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 15,
    frozenCount: 9,
    initialVisibleCount: 144,
    revealPerOrderComplete: 9,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 30,
    displayName: '第30关 廿四终宴',
    fruitIds: levelFruits(30),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 16,
    frozenCount: 9,
    initialVisibleCount: 146,
```

```
    revealPerOrderComplete: 9,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 31,
    displayName: '第31关 紫冠初席',
    fruitIds: levelFruits(31),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 16,
    frozenCount: 9,
    initialVisibleCount: 148,
    revealPerOrderComplete: 10,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 32,
    displayName: '第32关 油绿奶香',
    fruitIds: levelFruits(32),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 17,
    frozenCount: 9,
    initialVisibleCount: 150,
    revealPerOrderComplete: 10,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 33,
    displayName: '第33关 红宝石雨',
    fruitIds: levelFruits(33),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 17,
    frozenCount: 10,
    initialVisibleCount: 152,
    revealPerOrderComplete: 10,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 34,
    displayName: '第34关 蜜瓜弯月',
    fruitIds: levelFruits(34),
```

```
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 18,
    frozenCount: 10,
    initialVisibleCount: 154,
    revealPerOrderComplete: 10,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 35,
    displayName: '第35关 莲雾粉雾',
    fruitIds: levelFruits(35),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 18,
    frozenCount: 10,
    initialVisibleCount: 156,
    revealPerOrderComplete: 10,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 36,
    displayName: '第36关 回甘青果',
    fruitIds: levelFruits(36),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 18,
    frozenCount: 11,
    initialVisibleCount: 158,
    revealPerOrderComplete: 10,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 37,
    displayName: '第37关 芭乐餐车',
    fruitIds: levelFruits(37),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 19,
    frozenCount: 11,
    initialVisibleCount: 160,
    revealPerOrderComplete: 11,
    plateLanesInitial: 2,
```

```
    ...allTools,
  },
  {
    levelNumber: 38,
    displayName: '第38关 木瓜暖盅',
    fruitIds: levelFruits(38),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 19,
    frozenCount: 11,
    initialVisibleCount: 162,
    revealPerOrderComplete: 11,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 39,
    displayName: '第39关 无花蜜宴',
    fruitIds: levelFruits(39),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 20,
    frozenCount: 12,
    initialVisibleCount: 164,
    revealPerOrderComplete: 11,
    plateLanesInitial: 2,
    ...allTools,
  },
  {
    levelNumber: 40,
    displayName: '第40关 杏光终宴',
    fruitIds: levelFruits(40),
    copiesPerFruit: POST_15_COPIES_PER_FRUIT,
    orderTarget: 3,
    bufferSize: 5,
    iceCount: 20,
    frozenCount: 12,
    initialVisibleCount: 166,
    revealPerOrderComplete: 11,
    plateLanesInitial: 2,
    ...allTools,
  },
];
export const BOWL_LEVEL_COUNT = BOWL_LEVELS.length;
export function getBowlLevelDef(index: number): BowlLevelDef {
  const clamped = Math.max(0, Math.min(index, BOWL_LEVELS.length - 1));
  return BOWL_LEVELS[clamped]!;
}
```

```
/** 相对所有前序关卡新出现的食材（用于过关「解锁食材」展示；第 1 关为全部本关食材） */
export function getNewFruitsIntroducedInLevel(levelIndex: number): FruitId[] {
  const def = getBowlLevelDef(levelIndex);
  if (levelIndex <= 0) {
    return def.fruitIds.slice();
  }
  const seen = new Set<FruitId>();
  for (let i = 0; i < levelIndex; i += 1) {
    for (const id of getBowlLevelDef(i).fruitIds) {
      seen.add(id);
    }
  }
  return def.fruitIds.filter((id) => !seen.has(id));
}
import { BOWL_IMAGES_ROOT } from '@/config/bowlAssets';
export const BOWL_SOUP_KEYS = [
  'milk',
  'berry_tomato',
  'matcha',
  'mango_coconut',
  'taro_purple',
  'cocoa',
  'avocado_yogurt',
  'pomegranate_soda',
] as const;
export const BOWL_RIM_KEYS = [
  'crystal',
  'pink_ceramic',
  'mint_glass',
  'sesame_clay',
  'gold_porcelain',
  'star_glass',
  'coral_shell',
  'honey_amber_glass',
] as const;
export type BowlSoupKey = (typeof BOWL_SOUP_KEYS)[number];
export type BowlRimKey = (typeof BOWL_RIM_KEYS)[number];
export const DEFAULT_BOWL_SOUP_KEY: BowlSoupKey = 'milk';
export const DEFAULT_BOWL_RIM_KEY: BowlRimKey = 'crystal';
export const BOWL_SOUP_ASSETS: Record<BowlSoupKey, string> = {
  milk: `${BOWL_IMAGES_ROOT}/bowl_soup_milk.png`,
  berry_tomato: `${BOWL_IMAGES_ROOT}/bowl_soup_berry_tomato.png`,
  matcha: `${BOWL_IMAGES_ROOT}/bowl_soup_matcha.png`,
  mango_coconut: `${BOWL_IMAGES_ROOT}/bowl_soup_mango_coconut.png`,
  taro_purple: `${BOWL_IMAGES_ROOT}/bowl_soup_taro_purple.png`,
  cocoa: `${BOWL_IMAGES_ROOT}/bowl_soup_cocoa.png`,
  avocado_yogurt: `${BOWL_IMAGES_ROOT}/bowl_soup_avocado_yogurt.png`,
  pomegranate_soda: `${BOWL_IMAGES_ROOT}/bowl_soup_pomegranate_soda.png`,
};
export const BOWL_RIM_ASSETS: Record<BowlRimKey, string> = {
```

```
  crystal: `${BOWL_IMAGES_ROOT}/bowl_crystal_rim.png`,
  pink_ceramic: `${BOWL_IMAGES_ROOT}/bowl_rim_pink_ceramic.png`,
  mint_glass: `${BOWL_IMAGES_ROOT}/bowl_rim_mint_glass.png`,
  sesame_clay: `${BOWL_IMAGES_ROOT}/bowl_rim_sesame_clay.png`,
  gold_porcelain: `${BOWL_IMAGES_ROOT}/bowl_rim_gold_porcelain.png`,
  star_glass: `${BOWL_IMAGES_ROOT}/bowl_rim_star_glass.png`,
  coral_shell: `${BOWL_IMAGES_ROOT}/bowl_rim_coral_shell.png`,
  honey_amber_glass: `${BOWL_IMAGES_ROOT}/bowl_rim_honey_amber_glass.png`,
};
export const BOWL_SOUP_UNLOCKS: Array<{ levelNumber: number; key: BowlSoupKey; label: string }> = [
  { levelNumber: 1, key: 'milk', label: '奶白汤' },
  { levelNumber: 4, key: 'berry_tomato', label: '草莓番茄红汤' },
  { levelNumber: 9, key: 'matcha', label: '抹茶绿汤' },
  { levelNumber: 14, key: 'mango_coconut', label: '芒果椰乳黄汤' },
  { levelNumber: 20, key: 'taro_purple', label: '芋泥紫汤' },
  { levelNumber: 26, key: 'cocoa', label: '可可巧克力汤' },
  { levelNumber: 32, key: 'avocado_yogurt', label: '牛油果酸奶绿汤' },
  { levelNumber: 36, key: 'pomegranate_soda', label: '石榴气泡红汤' },
];
export const BOWL_RIM_UNLOCKS: Array<{ levelNumber: number; key: BowlRimKey; label: string }> = [
  { levelNumber: 1, key: 'crystal', label: '水晶碗' },
  { levelNumber: 6, key: 'pink_ceramic', label: '樱粉陶瓷碗' },
  { levelNumber: 11, key: 'mint_glass', label: '青釉玻璃碗' },
  { levelNumber: 17, key: 'sesame_clay', label: '黑芝麻陶土碗' },
  { levelNumber: 23, key: 'gold_porcelain', label: '金边白瓷碗' },
  { levelNumber: 29, key: 'star_glass', label: '星空玻璃碗' },
  { levelNumber: 34, key: 'coral_shell', label: '珊瑚贝壳碗' },
  { levelNumber: 39, key: 'honey_amber_glass', label: '蜂蜜琥珀玻璃碗' },
];
export interface BowlSkinUnlock {
  kind: 'soup' | 'bowl';
  key: BowlSoupKey | BowlRimKey;
  label: string;
  textureKey: string;
}
export function getBowlSoupKeyForLevel(levelNumber: number): BowlSoupKey {
  let key = DEFAULT_BOWL_SOUP_KEY;
  for (const unlock of BOWL_SOUP_UNLOCKS) {
    if (levelNumber >= unlock.levelNumber) {
      key = unlock.key;
    }
  }
  return key;
}
export function getBowlRimKeyForLevel(levelNumber: number): BowlRimKey {
  let key = DEFAULT_BOWL_RIM_KEY;
  for (const unlock of BOWL_RIM_UNLOCKS) {
    if (levelNumber >= unlock.levelNumber) {
      key = unlock.key;
    }
```

```
  }
  return key;
}
export function getBowlSkinUnlocksInLevel(levelNumber: number): BowlSkinUnlock[] {
  return [
    ...BOWL_SOUP_UNLOCKS.filter((unlock) => unlock.levelNumber === levelNumber).map((unlock) => ({
      kind: 'soup' as const,
      key: unlock.key,
      label: unlock.label,
      textureKey: `bowl_soup_${unlock.key}`,
    })),
    ...BOWL_RIM_UNLOCKS.filter((unlock) => unlock.levelNumber === levelNumber).map((unlock) => ({
      kind: 'bowl' as const,
      key: unlock.key,
      label: unlock.label,
      textureKey: `bowl_rim_${unlock.key}`,
    })),
  ];
}
import { BOWL_THEME_IMAGES_ROOT } from '@/config/bowlAssets';
export const BOWL_THEME_KEYS = [
  'mint_waterpark',
  'tropical_fruit_stand',
  'pool_ice_drink',
  'seaside_night_market',
  'garden_picnic',
  'fruit_restaurant',
  'beach_picnic',
] as const;
export type BowlThemeKey = (typeof BOWL_THEME_KEYS)[number];
export interface BowlThemeDef {
  key: BowlThemeKey;
  label: string;
  backdropAsset: string;
  bgTop: number;
  bgBottom: number;
  header: number;
  headerAccent: number;
  board: number;
  boardAccent: number;
  hudOuter: number;
  hudInner: number;
  hudStroke: number;
  hudText: number;
  orderBubble: number;
  orderBubbleStroke: number;
  progressFill: number;
  progressFillHi: number;
  slotTint: number;
}
```

```
export const BOWL_THEMES: Record<BowlThemeKey, BowlThemeDef> = {
  mint_waterpark: {
    key: 'mint_waterpark',
    label: '薄荷水上乐园',
    backdropAsset: `${BOWL_THEME_IMAGES_ROOT}/bowl_theme_mint_waterpark.jpg`,
    bgTop: 0xcff9df,
    bgBottom: 0xeaffcf,
    header: 0x90eadc,
    headerAccent: 0x4cc8bd,
    board: 0xa8f3df,
    boardAccent: 0x60d2c9,
    hudOuter: 0x248c91,
    hudInner: 0x3bb7b6,
    hudStroke: 0xd9fff2,
    hudText: 0xf5fff3,
    orderBubble: 0xffffee,
    orderBubbleStroke: 0x49b8a8,
    progressFill: 0x42d8c8,
    progressFillHi: 0xbffff2,
    slotTint: 0xd7fff5,
  },
  tropical_fruit_stand: {
    key: 'tropical_fruit_stand',
    label: '热带果摊',
    backdropAsset: `${BOWL_THEME_IMAGES_ROOT}/bowl_theme_tropical_fruit_stand.jpg`,
    bgTop: 0xfff0b8,
    bgBottom: 0xffffe8,
    header: 0xd9a961,
    headerAccent: 0x75b85b,
    board: 0xe9bd77,
    boardAccent: 0xb98343,
    hudOuter: 0x5d3c1f,
    hudInner: 0x8f5d2f,
    hudStroke: 0xffde75,
    hudText: 0xfff6bd,
    orderBubble: 0xffffef,
    orderBubbleStroke: 0x8d6234,
    progressFill: 0x71cf65,
    progressFillHi: 0xdfffb1,
    slotTint: 0xfff1c9,
  },
  pool_ice_drink: {
    key: 'pool_ice_drink',
    label: '泳池冰饮',
    backdropAsset: `${BOWL_THEME_IMAGES_ROOT}/bowl_theme_pool_ice_drink.jpg`,
    bgTop: 0xa4f2ff,
    bgBottom: 0xe6fff7,
    header: 0x7addec,
    headerAccent: 0x2ba9d2,
    board: 0xb8f4f1,
```

```
    boardAccent: 0x58cfdc,
    hudOuter: 0x217f9a,
    hudInner: 0x3bb5c8,
    hudStroke: 0xd9fbff,
    hudText: 0xf3ffff,
    orderBubble: 0xf7ffff,
    orderBubbleStroke: 0x4fbfd4,
    progressFill: 0x38c7f1,
    progressFillHi: 0xc9f7ff,
    slotTint: 0xdbfbff,
  },
  garden_picnic: {
    key: 'garden_picnic',
    label: '花园野餐',
    backdropAsset: `${BOWL_THEME_IMAGES_ROOT}/bowl_theme_garden_picnic.jpg`,
    bgTop: 0xfff4e8,
    bgBottom: 0xf8fff0,
    header: 0xc8e8b0,
    headerAccent: 0x8bc98a,
    board: 0xe8d8b0,
    boardAccent: 0xb8c98a,
    hudOuter: 0x5a6b3f,
    hudInner: 0x7d8f6a,
    hudStroke: 0xfff6d8,
    hudText: 0xfff8ef,
    orderBubble: 0xfffff2,
    orderBubbleStroke: 0x8d9a7a,
    progressFill: 0x7fcf8a,
    progressFillHi: 0xe8ffd8,
    slotTint: 0xfff8ef,
  },
  seaside_night_market: {
    key: 'seaside_night_market',
    label: '海边夜市',
    backdropAsset: `${BOWL_THEME_IMAGES_ROOT}/bowl_theme_seaside_night_market.jpg`,
    bgTop: 0x3e426c,
    bgBottom: 0xffba89,
    header: 0x4b3c67,
    headerAccent: 0xffc46b,
    board: 0xd8876b,
    boardAccent: 0x714e68,
    hudOuter: 0x33263e,
    hudInner: 0x5b3c61,
    hudStroke: 0xffd180,
    hudText: 0xfff0c7,
    orderBubble: 0xffefd9,
    orderBubbleStroke: 0x8d5472,
    progressFill: 0xff9f7a,
    progressFillHi: 0xffe0a0,
    slotTint: 0xffe0d4,
```

```
  },
  fruit_restaurant: {
    key: 'fruit_restaurant',
    label: '果味餐厅',
    backdropAsset: `${BOWL_THEME_IMAGES_ROOT}/bowl_theme_fruit_restaurant.jpg`,
    bgTop: 0xfff0e8,
    bgBottom: 0xfffff5,
    header: 0xe8c4a8,
    headerAccent: 0xc98a6a,
    board: 0xf0d4b8,
    boardAccent: 0xb88868,
    hudOuter: 0x6b4030,
    hudInner: 0x9a6048,
    hudStroke: 0xffe8d0,
    hudText: 0xfff8ef,
    orderBubble: 0xfffff5,
    orderBubbleStroke: 0xa07058,
    progressFill: 0xe89070,
    progressFillHi: 0xffe0c8,
    slotTint: 0xfff0e8,
  },
  beach_picnic: {
    key: 'beach_picnic',
    label: '沙滩野餐',
    backdropAsset: `${BOWL_THEME_IMAGES_ROOT}/bowl_theme_beach_picnic.jpg`,
    bgTop: 0xd4f4ff,
    bgBottom: 0xfff8e8,
    header: 0x8ad4e8,
    headerAccent: 0x4ab8d8,
    board: 0xc8e8f0,
    boardAccent: 0x68b8d0,
    hudOuter: 0x2a6888,
    hudInner: 0x48a0b8,
    hudStroke: 0xe8ffff,
    hudText: 0xf5ffff,
    orderBubble: 0xf7ffff,
    orderBubbleStroke: 0x58a8c8,
    progressFill: 0x48c8e8,
    progressFillHi: 0xc8f0ff,
    slotTint: 0xe8f8ff,
  },
};
export const DEFAULT_BOWL_THEME_KEY: BowlThemeKey = 'tropical_fruit_stand';
const THEME_UNLOCKS: Array<{ levelNumber: number; key: BowlThemeKey }> = [
  { levelNumber: 1, key: 'tropical_fruit_stand' },
  { levelNumber: 4, key: 'mint_waterpark' },
  { levelNumber: 7, key: 'pool_ice_drink' },
  { levelNumber: 10, key: 'garden_picnic' },
  { levelNumber: 13, key: 'seaside_night_market' },
  { levelNumber: 16, key: 'fruit_restaurant' },
```

```
  { levelNumber: 19, key: 'beach_picnic' },
  { levelNumber: 22, key: 'tropical_fruit_stand' },
  { levelNumber: 25, key: 'mint_waterpark' },
  { levelNumber: 28, key: 'pool_ice_drink' },
  { levelNumber: 31, key: 'garden_picnic' },
  { levelNumber: 34, key: 'seaside_night_market' },
  { levelNumber: 37, key: 'fruit_restaurant' },
  { levelNumber: 40, key: 'beach_picnic' },
];
export function getBowlThemeKeyForLevel(levelNumber: number): BowlThemeKey {
  let key = DEFAULT_BOWL_THEME_KEY;
  for (const unlock of THEME_UNLOCKS) {
    if (levelNumber >= unlock.levelNumber) {
      key = unlock.key;
    }
  }
  return key;
}
export function getBowlTheme(key: BowlThemeKey): BowlThemeDef {
  return BOWL_THEMES[key] ?? BOWL_THEMES[DEFAULT_BOWL_THEME_KEY];
}
import type { FruitId } from '@/config/fruits';
export const DAILY_LIMITED_MAX_FRUIT_TYPES = 19;
export const DAILY_LIMITED_MIN_STACK_CARDS = 210;
/** 每日限定每局每种道具（洗牌 / 撤销 / 抬起）各自可用次数上限 */
export const DAILY_LIMITED_TOOL_USES_PER_ROUND = 3;
export interface DailyDrinkRecipe {
  readonly title: string;
  readonly intro: string;
  readonly steps: readonly string[];
}
export interface DailyThemeTargetDef {
  readonly fruitId: FruitId;
  readonly requiredCount: number;
  readonly cardCopies: number;
}
export interface DailyThemeRecipeCardDef {
  readonly textureKey: string;
  readonly path: string;
  readonly catalogTitle: string;
  readonly catalogSubtitle: string;
  readonly shareTitle: string;
}
export interface DailyThemeLevelDef {
  readonly dayOfMonth: number;
  readonly themeId: string;
  readonly themeName: string;
  readonly drinkName: string;
  readonly targets: readonly DailyThemeTargetDef[];
  /** 堆叠区总卡数。每日限定必须保持满屏堆叠难度，只能大于等于 DAILY_LIMITED_MIN_STACK_CARDS。 */
```

```
  readonly totalStackCards?: number;
  readonly fruitIds: readonly FruitId[];
  readonly positioningText: string;
  readonly recipeUnlock: DailyDrinkRecipe;
  readonly recipeCard: DailyThemeRecipeCardDef;
  readonly bufferSize: number;
  readonly toolCounts: {
    readonly shuffle: number;
    readonly undo: number;
    readonly lift: number;
  };
  readonly layoutSeed: number;
}
export function getDailyLimitedPlayableFruitIds(level: DailyThemeLevelDef): readonly FruitId[] {
  const result: FruitId[] = [];
  const seen = new Set<FruitId>();
  const push = (fruitId: FruitId): void => {
    if (seen.has(fruitId) || result.length >= DAILY_LIMITED_MAX_FRUIT_TYPES) {
      return;
    }
    seen.add(fruitId);
    result.push(fruitId);
  };
  // 目标水果必须优先保留；剩余名额再补干扰水果，保证实际发牌种类不超过 19。
  for (const target of level.targets) {
    push(target.fruitId);
  }
  for (const fruitId of level.fruitIds) {
    push(fruitId);
  }
  return result;
}
const DAILY_LIMITED_COMMON_FRUITS: readonly FruitId[] = [
  'pineapple',
  'apple',
  'avocado',
  'banana',
  'bayberry',
  'blueberry',
  'cantaloupe',
  'cherry_tomato',
  'cucumber',
  'dragonfruit',
  'emblic',
  'grape_green',
  'grapefruit',
  'guava',
  'kumquat',
  'lemon',
  'lily_bulb',
```

```
    session_key: typeof data.session_key === 'string' ? data.session_key : '',
  };
}
async function ttCode2Openid(code) {
  const appid = getPlatformCredential('tt', 'APPID');
  const secret = getPlatformCredential('tt', 'SECRET');
  if (!appid || !secret) throw httpError(500, 'NO_TT_CFG', `${gameKeyUpper()}_TT_APPID/${gameKeyU...
  if (!code) throw httpError(400, 'NO_CODE', 'dy code 缺失');
  const data = await httpPostJson('https://developer.toutiao.com/api/apps/v2/jscode2session', { a...
  if (!data || data.err_no !== 0 || !data.data || !data.data.openid) {
    throw httpError(401, 'TT_LOGIN_FAIL', `dy code2session 失败: ${JSON.stringify(data || {})}`);
  }
  return data.data.openid;
}
function httpGetJson(url) {
  return httpRequestJson(url, 'GET');
}
function httpPostJson(url, body) {
  return httpRequestJson(url, 'POST', body);
}
function httpRequestJson(url, method, body) {
  if (typeof fetch !== 'function') {
    return Promise.reject(httpError(500, 'NO_FETCH', '当前 Node 运行时不支持 fetch，请使用 Node 18+'));
  }
  const opts = { method, headers: { 'Content-Type': 'application/json' } };
  if (body !== undefined) {
    opts.body = JSON.stringify(body);
  }
  return fetch(url, opts).then(async (res) => {
    const text = await res.text();
    try {
      return JSON.parse(text);
    } catch (_) {
      return { _raw: text };
    }
  });
}
module.exports = {
  handleLogin,
  requireUser,
};
const DEFAULT_GAME_KEY = 'hotpot';
const DEFAULT_TTL_SEC = 7 * 24 * 3600;
const DEFAULT_MAX_BYTES = 256 * 1024;
const DEFAULT_RANK_LIST_LIMIT = 50;
const DEFAULT_RANK_LIST_MAX_LIMIT = 100;
const DEFAULT_RANK_MINE_SCAN_LIMIT = 1000;
const DEFAULT_RANK_BOWL_MAX_LEVEL = 40;
const DEFAULT_RANK_FRUIT_MAX_SCORE = 9999999;
const DEFAULT_LEVEL_PASS_RATE_COLLECTION = 'hotpot_public_level_pass_rates';
```

```
function getGameKey() {
  const v = String(process.env.GAME_KEY || '').trim().toLowerCase();
  if (!v) return DEFAULT_GAME_KEY;
  if (!/^[a-z][a-z0-9_\-]{0,31}$/.test(v)) {
    throw new Error(`非法 GAME_KEY: ${v}`);
  }
  return v;
}
function gameKeyUpper() {
  return getGameKey().toUpperCase().replace(/[^A-Z0-9]/g, '_');
}
function readEnvPrefer(...keys) {
  for (const key of keys) {
    const value = process.env[key];
    if (value !== undefined && value !== null && String(value).length > 0) {
      return String(value);
    }
  }
  return '';
}
function getCollectionName(suffix) {
  const overrideKey = `${gameKeyUpper()}_${suffix.toUpperCase()}_COLLECTION`;
  const override = process.env[overrideKey];
  if (override) {
    return String(override);
  }
  return `${getGameKey()}_${suffix}`;
}
function getJwtSecret() {
  return readEnvPrefer(`${gameKeyUpper()}_JWT_SECRET`);
}
function getTtlSec() {
  const raw = readEnvPrefer(`${gameKeyUpper()}_TOKEN_TTL_SEC`);
  const v = Number(raw);
  return Number.isFinite(v) && v > 0 ? Math.floor(v) : DEFAULT_TTL_SEC;
}
function getMaxBytes() {
  const raw = readEnvPrefer(`${gameKeyUpper()}_SAVE_MAX_BYTES`);
  const v = Number(raw);
  return Number.isFinite(v) && v > 0 ? Math.floor(v) : DEFAULT_MAX_BYTES;
}
function getPlatformCredential(platform, field) {
  const upper = gameKeyUpper();
  const platformUpper = platform.toUpperCase();
  return readEnvPrefer(`${upper}_${platformUpper}_${field}`);
}
function readPositiveIntEnv(key, fallback) {
  const v = Number(readEnvPrefer(`${gameKeyUpper()}_${key}`));
  return Number.isFinite(v) && v > 0 ? Math.floor(v) : fallback;
}
```

```
function getRankListLimit() {
  return readPositiveIntEnv('RANK_LIST_LIMIT', DEFAULT_RANK_LIST_LIMIT);
}
function getRankListMaxLimit() {
  return readPositiveIntEnv('RANK_LIST_MAX_LIMIT', DEFAULT_RANK_LIST_MAX_LIMIT);
}
function getRankMineScanLimit() {
  return readPositiveIntEnv('RANK_MINE_SCAN_LIMIT', DEFAULT_RANK_MINE_SCAN_LIMIT);
}
function getRankBowlMaxLevel() {
  return readPositiveIntEnv('RANK_BOWL_MAX_LEVEL', DEFAULT_RANK_BOWL_MAX_LEVEL);
}
function getRankFruitMaxScore() {
  return readPositiveIntEnv('RANK_FRUIT_MAX_SCORE', DEFAULT_RANK_FRUIT_MAX_SCORE);
}
function getLevelPassRateCollectionName() {
  return readEnvPrefer(`${gameKeyUpper()}_LEVEL_PASS_RATE_COLLECTION`) || DEFAULT_LEVEL_PASS_RATE...
}
module.exports = {
  getGameKey,
  gameKeyUpper,
  getCollectionName,
  getJwtSecret,
  getTtlSec,
  getMaxBytes,
  getPlatformCredential,
  getRankListLimit,
  getRankListMaxLimit,
  getRankMineScanLimit,
  getRankBowlMaxLevel,
  getRankFruitMaxScore,
  getLevelPassRateCollectionName,
};
const tcb = require('@cloudbase/node-sdk');
const { getCollectionName, getLevelPassRateCollectionName } = require('./config');
let app = null;
function getApp() {
  if (app) return app;
  app = tcb.init({
    env: process.env.TCB_ENV || tcb.SYMBOL_CURRENT_ENV,
  });
  return app;
}
function getDb() {
  return getApp().database();
}
function getCollection() {
  return getDb().collection(getCollectionName('playerData'));
}
function getRankingCollection() {
```

```
  return getDb().collection(getCollectionName('rankings'));
}
function getLevelPassRateCollection() {
  return getDb().collection(getLevelPassRateCollectionName());
}
module.exports = {
  getApp,
  getDb,
  getCollection,
  getRankingCollection,
  getLevelPassRateCollection,
};
const crypto = require('crypto');
const { httpError } = require('./http');
const { getDb } = require('./db');
const { getCollectionName } = require('./config');
function getSessionCollection() {
  return getDb().collection(getCollectionName('wxSessions'));
}
async function upsertWxSession(userId, sessionKey) {
  if (!sessionKey) {
    return;
  }
  const col = getSessionCollection();
  const now = Date.now();
  const existingRes = await col.where({ userId }).limit(1).get();
  const existing = (existingRes && Array.isArray(existingRes.data) && existingRes.data[0]) || null;
  if (existing && existing._id) {
    await col.doc(existing._id).update({
      sessionKey,
      updatedAt: now,
    });
    return;
  }
  await col.add({
    userId,
    sessionKey,
    updatedAt: now,
  });
}
async function readWxSessionKey(userId) {
  const col = getSessionCollection();
  const res = await col.where({ userId }).limit(1).get();
  const doc = (res && Array.isArray(res.data) && res.data[0]) || null;
  return doc && typeof doc.sessionKey === 'string' ? doc.sessionKey : '';
}
function decryptGameClubPayload(sessionKey, encryptedData, iv) {
  if (!sessionKey || !encryptedData || !iv) {
    throw httpError(400, 'BAD_GAME_CLUB_PAYLOAD', 'encryptedData / iv / sessionKey 缺失');
  }
```

```
  try {
    const decipher = crypto.createDecipheriv(
      'aes-128-cbc',
      Buffer.from(sessionKey, 'base64'),
      Buffer.from(iv, 'base64'),
    );
    decipher.setAutoPadding(true);
    const decoded = Buffer.concat([
      decipher.update(Buffer.from(encryptedData, 'base64')),
      decipher.final(),
    ]).toString('utf8');
    return JSON.parse(decoded);
  } catch (error) {
    throw httpError(400, 'DECRYPT_FAIL', error && error.message ? error.message : '游戏圈数据解密失败');
  }
}
async function handleDecrypt(req) {
  // Lazy-load to avoid a circular dependency with auth.js, which stores wx sessions here.
  const { requireUser } = require('./auth');
  const { userId } = requireUser(req);
  const body = req.body || {};
  const encryptedData = String(body.encryptedData || '').trim();
  const iv = String(body.iv || '').trim();
  const sessionKey = await readWxSessionKey(userId);
  if (!sessionKey) {
    throw httpError(400, 'NO_WX_SESSION', '微信 session 未就绪，请重新进入游戏');
  }
  return decryptGameClubPayload(sessionKey, encryptedData, iv);
}
module.exports = {
  upsertWxSession,
  handleDecrypt,
};
const { getGameKey } = require('./config');
const CORS_HEADERS = {
  'Access-Control-Allow-Origin': '*',
  'Access-Control-Allow-Methods': 'GET,POST,OPTIONS',
  'Access-Control-Allow-Headers': 'Content-Type,Authorization',
  'Access-Control-Max-Age': '86400',
};
function respond(statusCode, body, extraHeaders = {}) {
  return {
    statusCode,
    headers: {
      'Content-Type': 'application/json; charset=utf-8',
      ...CORS_HEADERS,
      ...extraHeaders,
    },
    body: typeof body === 'string' ? body : JSON.stringify(body),
    isBase64Encoded: false,
```

```
  };
}
function preflight() {
  return {
    statusCode: 204,
    headers: { ...CORS_HEADERS },
    body: '',
    isBase64Encoded: false,
  };
}
function parseEvent(event) {
  event = event || {};
  if (event.httpMethod) {
    const method = String(event.httpMethod).toUpperCase();
    let rawBody = event.body || '';
    if (event.isBase64Encoded && rawBody) {
      try {
        rawBody = Buffer.from(rawBody, 'base64').toString('utf8');
      } catch (_) {}
    }
    let body = {};
    if (rawBody) {
      try {
        body = JSON.parse(rawBody);
      } catch (_) {
        body = {};
      }
    }
    return {
      method,
      path: normalizePath(event.path || '/'),
      body,
      headers: lowercaseHeaders(event.headers || {}),
      query: event.queryStringParameters || {},
      raw: event,
    };
  }
  const action = String(event.action || '').replace(/^\/+/, '');
  return {
    method: 'POST',
    path: action ? `/${action}` : '/',
    body: event.body || {},
    headers: lowercaseHeaders(event.headers || {}),
    query: {},
    raw: event,
  };
}
function normalizePath(path) {
  if (!path) return '/';
  let p = String(path);
```

```
  if (!p.startsWith('/')) p = `/${p}`;
  const apiPrefix = `${getGameKey()}-api`;
  p = p.replace(new RegExp(`^/(?:${escapeRegExp(apiPrefix)})(?=/|$)`), '');
  if (p === '') p = '/';
  if (p.length > 1 && p.endsWith('/')) p = p.slice(0, -1);
  return p;
 }
 function lowercaseHeaders(headers) {
  const out = {};
  for (const key of Object.keys(headers || {})) {
   out[key.toLowerCase()] = headers[key];
  }
  return out;
 }
 function httpError(status, code, message) {
  const err = new Error(message || code);
  err.status = status;
  err.code = code;
  return err;
 }
 function escapeRegExp(text) {
  return String(text).replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
 }
 module.exports = {
  respond,
  preflight,
  parseEvent,
  httpError,
 };
 const { requireUser } = require('./auth');
 const { getLevelPassRateCollection } = require('./db');
 const { httpError } = require('./http');
 const MODE_BOWL = 'bowl';
 const WINDOW_DAYS = 30;
 const DOC_ID = `${MODE_BOWL}_${WINDOW_DAYS}d_latest`;
 function normalizeMode(value) {
  const mode = String(value || MODE_BOWL).trim();
  if (mode !== MODE_BOWL) {
   throw httpError(400, 'BAD_MODE', `unsupported mode_key: ${mode}`);
  }
  return mode;
 }
 function normalizeWindowDays(value) {
  const days = Number(value || WINDOW_DAYS);
  if (!Number.isFinite(days) || Math.floor(days) !== WINDOW_DAYS) {
   throw httpError(400, 'BAD_WINDOW_DAYS', `unsupported window_days: ${value}`);
  }
  return WINDOW_DAYS;
 }
 function publicSnapshot(doc) {
```

```
  return {
    game_key: String(doc.game_key || 'hotpot'),
    mode_key: MODE_BOWL,
    window_days: WINDOW_DAYS,
    window_start_date: String(doc.window_start_date || ''),
    window_end_date: String(doc.window_end_date || ''),
    computed_at: Number(doc.computed_at) || 0,
    levels: Array.isArray(doc.levels) ? doc.levels.map(publicLevel).filter(Boolean) : [],
  };
}
function publicLevel(item) {
  const levelId = Number(item && item.level_id);
  if (!Number.isFinite(levelId) || levelId <= 0) {
    return null;
  }
  const passRate = Number(item.pass_rate);
  return {
    level_id: Math.floor(levelId),
    pass_rate: Number.isFinite(passRate) ? Math.max(0, Math.min(1, passRate)) : 0,
    start_users: Math.max(0, Math.floor(Number(item.start_users) || 0)),
    clear_users: Math.max(0, Math.floor(Number(item.clear_users) || 0)),
    started_and_cleared_users: Math.max(0, Math.floor(Number(item.started_and_cleared_users) || 0)),
    is_sample_low: item.is_sample_low === true || item.is_sample_low === 1,
  };
}
async function handleList(req) {
  requireUser(req);
  const body = req.body || {};
  normalizeMode(body.mode_key);
  normalizeWindowDays(body.window_days);
  const res = await getLevelPassRateCollection().doc(DOC_ID).get();
  const doc = res && (Array.isArray(res.data) ? res.data[0] : res.data);
  if (!doc) {
    throw httpError(404, 'LEVEL_PASS_RATE_NOT_READY', '关卡通关率快照尚未生成');
  }
  return publicSnapshot(doc);
}
module.exports = {
  handleList,
};
const crypto = require('crypto');
const { requireUser } = require('./auth');
const { getRankingCollection } = require('./db');
const { httpError } = require('./http');
const {
  getRankBowlMaxLevel,
  getRankFruitMaxScore,
  getRankListLimit,
  getRankListMaxLimit,
  getRankMineScanLimit,
```

```
} = require('./config');
const BOARD_BOWL = 'bowl_progress';
const BOARD_FRUIT = 'fruit_best';
const BOARDS = new Set([BOARD_BOWL, BOARD_FRUIT]);
const RANK_SUBMIT_BLOCKED_USER_IDS = new Set([
  // GM 测试账号：不再写入 / 更新排行榜数据，避免测试进度污染线上榜单。
  'wx:oB0xx3SeJgkkU0_ONokPrzvFljrE',
]);
function normalizeBoard(value) {
  const board = String(value || '').trim();
  if (!BOARDS.has(board)) {
    throw httpError(400, 'BAD_BOARD', `unsupported board: ${board}`);
  }
  return board;
}
function normalizeNonNegativeInt(value, field) {
  const n = Number(value);
  if (!Number.isFinite(n) || n < 0) {
    throw httpError(400, `BAD_${field.toUpperCase()}`, `${field} 必须为非负整数`);
  }
  return Math.floor(n);
}
function normalizeLimit(value) {
  const fallback = getRankListLimit();
  const max = Math.max(1, getRankListMaxLimit());
  const n = Number(value);
  if (!Number.isFinite(n) || n <= 0) {
    return Math.min(fallback, max);
  }
  return Math.min(Math.floor(n), max);
}
function normalizeOffset(value) {
  const n = Number(value);
  return Number.isFinite(n) && n > 0 ? Math.floor(n) : 0;
}
function displayNameForUser(userId) {
  const digest = crypto.createHash('sha1').update(String(userId)).digest('hex');
  const suffix = String(parseInt(digest.slice(0, 8), 16) % 10000).padStart(4, '0');
  return `水果达人${suffix}`;
}
function sanitizeDisplayName(value, fallback) {
  const text = String(value || '').trim().replace(/[\r\n\t]/g, ' ').slice(0, 16);
  return text || fallback;
}
/**
 * 仅接受 http(s):// 开头、长度不超过 1024 的头像 URL；
 * 其余视为无效，回退到 fallback（一般是已有头像或空字符串）。
 */
function sanitizeAvatarUrl(value, fallback) {
  const text = String(value || '').trim();
```

```
  if (!text) {
    return fallback || '';
  }
  if (text.length > 1024) {
    return fallback || '';
  }
  if (!/^https?:\/\//i.test(text)) {
    return fallback || '';
  }
  return text;
}
function isBetterRecord(board, next, prev) {
  if (!prev) return true;
  if (board === BOARD_BOWL) {
    const prevBadge = bowlClearedLevel(prev);
    const nextBadge = Number(next.badgeLevel) || 0;
    return nextBadge > prevBadge;
  }
  return next.score > (Number(prev.score) || 0);
}
function bowlClearedLevel(doc) {
  if (!doc) return 0;
  if (doc.badgeLevel !== undefined && doc.badgeLevel !== null) {
    return Number(doc.badgeLevel) || 0;
  }
  // 极旧记录如果没有 badgeLevel，则回退到 level，避免真实上榜记录消失。
  return Number(doc.level) || 0;
}
function publicRecord(doc, userId, rank) {
  if (!doc) return null;
  const out = {
    rank: rank || null,
    board: doc.board,
    displayName: doc.displayName || displayNameForUser(doc.userId || ''),
    avatarUrl: doc.avatarUrl || '',
    isMe: !!userId && doc.userId === userId,
    updatedAt: Number(doc.updatedAt) || 0,
  };
  if (doc.board === BOARD_BOWL) {
    const clearedLevel = bowlClearedLevel(doc);
    out.level = clearedLevel;
    out.badgeLevel = clearedLevel;
  } else {
    out.score = Number(doc.score) || 0;
  }
  return out;
}
function orderedQuery(col, board) {
  let query = col.where({ board });
  if (board === BOARD_BOWL) {
```

```
    query = query.orderBy('badgeLevel', 'desc').orderBy('updatedAt', 'asc');
  } else {
    query = query.orderBy('score', 'desc').orderBy('updatedAt', 'asc');
  }
  return query;
}
async function findMineRank(col, board, userId) {
  const scanLimit = Math.max(getRankListMaxLimit(), getRankMineScanLimit());
  const res = await orderedQuery(col, board).limit(scanLimit).get();
  const list = (res && Array.isArray(res.data) ? res.data : []).filter(
    (item) => board !== BOARD_BOWL || bowlClearedLevel(item) > 0,
  );
  const index = list.findIndex((item) => item.userId === userId);
  if (index < 0) {
    return null;
  }
  return {
    rank: index + 1,
    doc: list[index],
  };
}
function buildSubmitRecord(board, body) {
  if (board === BOARD_BOWL) {
    const badgeLevel = normalizeNonNegativeInt(body.badgeLevel ?? body.level ?? 0, 'badgeLevel');
    const maxLevel = getRankBowlMaxLevel();
    if (badgeLevel > maxLevel) {
      throw httpError(400, 'LEVEL_TOO_HIGH', `badgeLevel 超出上限: ${badgeLevel} > ${maxLevel}`);
    }
    // level 与 badgeLevel 统一为「已通关关数」，兼容旧客户端仍传 level 的情况
    return { level: badgeLevel, badgeLevel, score: 0 };
  }
  const score = normalizeNonNegativeInt(body.score, 'score');
  const maxScore = getRankFruitMaxScore();
  if (score > maxScore) {
    throw httpError(400, 'SCORE_TOO_HIGH', `score 超出上限: ${score} > ${maxScore}`);
  }
  return { score, level: 0, badgeLevel: 0 };
}
async function handleSubmit(req) {
  const { userId, platform } = requireUser(req);
  const body = req.body || {};
  const board = normalizeBoard(body.board);
  const incoming = buildSubmitRecord(board, body);
  if (board === BOARD_BOWL && incoming.badgeLevel <= 0) {
    return {
      board,
      updated: false,
      reason: 'NO_PROGRESS',
      record: null,
    };
```

```
  }
  if (RANK_SUBMIT_BLOCKED_USER_IDS.has(userId)) {
    console.log(`[rank.submit] uid=${userId} board=${board} blocked GM test account`);
    return {
      board,
      updated: false,
      reason: 'GM_TEST_ACCOUNT',
      record: null,
    };
  }
  const col = getRankingCollection();
  const existingRes = await col.where({ userId, board }).limit(1).get();
  const existing = (existingRes && Array.isArray(existingRes.data) && existingRes.data[0]) || null;
  const now = Date.now();
  const displayName = sanitizeDisplayName(body.displayName, (existing && existing.displayName) ||...
  const avatarUrl = sanitizeAvatarUrl(body.avatarUrl, (existing && existing.avatarUrl) || '');
  // 排错关键日志：客户端真的把昵称/头像传上来了吗？sanitize 之后还剩什么？
  const rawAvatar = String(body.avatarUrl || '');
  console.log(
    `[rank.submit] uid=${userId} board=${board}` +
      ` rawDisplayName="${body.displayName || ''}" -> "${displayName}"` +
      ` rawAvatarUrl=${rawAvatar ? rawAvatar.slice(0, 48) + (rawAvatar.length > 48 ? '...' : '') ...
      ` sanitizedAvatar=${avatarUrl ? 'yes' : 'no'}` +
      ` hasExisting=${!!existing}`,
  );
  if (!isBetterRecord(board, incoming, existing)) {
    // 即使成绩不再更优，玩家也可能刚授权了新的微信昵称 / 头像 --
    // 这种情况下做轻量字段更新即可，不动 score/level/badge。
    if (existing && existing._id) {
      const sameName = existing.displayName === displayName;
      const sameAvatar = (existing.avatarUrl || '') === avatarUrl;
      if (!sameName || !sameAvatar) {
        const patch = { displayName, avatarUrl, updatedAt: now };
        await col.doc(existing._id).update(patch);
        console.log(
          `[rank.submit] uid=${userId} profile_update` +
            ` nameChanged=${!sameName} avatarChanged=${!sameAvatar}`,
        );
        return {
          board,
          updated: true,
          mode: 'profile_update',
          record: publicRecord({ ...existing, ...patch }, userId, null),
        };
      }
      console.log(`[rank.submit] uid=${userId} NOT_BETTER, profile already up to date`);
    } else {
      console.log(`[rank.submit] uid=${userId} NOT_BETTER, no existing doc`);
    }
    return {
```

```
      board,
      updated: false,
      reason: 'NOT_BETTER',
      record: publicRecord(existing, userId, null),
    };
  }
  const docData = {
    userId,
    board,
    platform,
    displayName,
    avatarUrl,
    score: incoming.score,
    level: incoming.level,
    badgeLevel: incoming.badgeLevel,
    createdAt: (existing && existing.createdAt) || now,
    updatedAt: now,
  };
  if (existing && existing._id) {
    await col.doc(existing._id).update(docData);
    console.log(`[rank.submit] uid=${userId} update score-record id=${existing._id}`);
    return { board, updated: true, mode: 'update', record: publicRecord({ ...existing, ...docData...
  }
  const addRes = await col.add(docData);
  console.log(`[rank.submit] uid=${userId} insert new score-record id=${addRes && (addRes.id || a...
  return {
    board,
    updated: true,
    mode: 'insert',
    record: publicRecord({ ...docData, _id: addRes && (addRes.id || addRes._id) }, userId, null),
  };
}
async function handleList(req) {
  const { userId } = requireUser(req);
  const body = req.body || {};
  const board = normalizeBoard(body.board);
  const limit = normalizeLimit(body.limit);
  const offset = normalizeOffset(body.offset);
  const col = getRankingCollection();
  const res = await orderedQuery(col, board).skip(offset).limit(limit).get();
  const rows = (res && Array.isArray(res.data) ? res.data : []).filter(
    (doc) => board !== BOARD_BOWL || bowlClearedLevel(doc) > 0,
  );
  const list = rows.map((doc, index) => publicRecord(doc, userId, offset + index + 1));
  const mineRank = await findMineRank(col, board, userId);
  const mine = mineRank ? publicRecord(mineRank.doc, userId, mineRank.rank) : null;
  return { board, limit, offset, list, mine };
}
async function handleMine(req) {
  const { userId } = requireUser(req);
```

```
  const board = normalizeBoard((req.body || {}).board);
  const col = getRankingCollection();
  const mineRank = await findMineRank(col, board, userId);
  return { board, mine: mineRank ? publicRecord(mineRank.doc, userId, mineRank.rank) : null };
}
module.exports = {
  BOARD_BOWL,
  BOARD_FRUIT,
  handleSubmit,
  handleList,
  handleMine,
};
const { httpError } = require('./http');
const { requireUser } = require('./auth');
const { getCollection } = require('./db');
const { getMaxBytes } = require('./config');
async function handlePull(req) {
  const { userId, platform } = requireUser(req);
  const col = getCollection();
  const res = await col.where({ userId }).limit(1).get();
  const doc = (res && Array.isArray(res.data) && res.data[0]) || null;
  if (!doc) {
    return {
      userId,
      platform,
      exists: false,
      schemaVersion: 0,
      updatedAt: 0,
      payload: {},
      payloadKeys: [],
    };
  }
  return {
    userId,
    platform,
    exists: true,
    schemaVersion: doc.schemaVersion || 0,
    updatedAt: doc.updatedAt || 0,
    payload: doc.payload || {},
    payloadKeys: Array.isArray(doc.payloadKeys) ? doc.payloadKeys : Object.keys(doc.payload || {}),
    clientFingerprint: doc.clientFingerprint || '',
  };
}
async function handlePush(req) {
  const { userId, platform } = requireUser(req);
  const body = req.body || {};
  const schemaVersion = Number(body.schemaVersion);
  const updatedAt = Number(body.updatedAt);
  const baseRemoteUpdatedAt = Number(body.baseRemoteUpdatedAt || 0);
  const payload = body.payload;
```

```
const clientFingerprint = String(body.clientFingerprint || '').slice(0, 200);
const force = body.force === true;
if (!Number.isFinite(schemaVersion) || schemaVersion <= 0) {
  throw httpError(400, 'BAD_SCHEMA', 'schemaVersion 非法');
}
if (!Number.isFinite(updatedAt) || updatedAt <= 0) {
  throw httpError(400, 'BAD_UPDATED_AT', 'updatedAt 非法');
}
if (!Number.isFinite(baseRemoteUpdatedAt) || baseRemoteUpdatedAt < 0) {
  throw httpError(400, 'BAD_BASE_REMOTE_UPDATED_AT', 'baseRemoteUpdatedAt 非法');
}
if (!payload || typeof payload !== 'object' || Array.isArray(payload)) {
  throw httpError(400, 'BAD_PAYLOAD', 'payload 必须是 object');
}
const size = Buffer.byteLength(JSON.stringify(payload), 'utf8');
const maxBytes = getMaxBytes();
if (size > maxBytes) {
  throw httpError(413, 'PAYLOAD_TOO_LARGE', `payload 超限: ${size}B > ${maxBytes}B`);
}
const payloadKeys = [];
for (const key of Object.keys(payload)) {
  if (typeof payload[key] !== 'string') {
    throw httpError(400, 'BAD_PAYLOAD_VALUE', `payload[${key}] 必须是字符串`);
  }
  payloadKeys.push(key);
}
const col = getCollection();
const existingRes = await col.where({ userId }).limit(1).get();
const existing = (existingRes && Array.isArray(existingRes.data) && existingRes.data[0]) || null;
if (existing && !force) {
  const prevUpdatedAt = Number(existing.updatedAt) || 0;
  if (updatedAt < prevUpdatedAt || baseRemoteUpdatedAt < prevUpdatedAt) {
    throw Object.assign(
      httpError(409, 'STALE_UPDATE', `服务端已有更新版本 remote=${prevUpdatedAt} > local=${updatedAt}, b...
      {
        data: {
          remote: {
            schemaVersion: existing.schemaVersion || 0,
            updatedAt: prevUpdatedAt,
            payload: existing.payload || {},
            payloadKeys: Array.isArray(existing.payloadKeys)
              ? existing.payloadKeys
              : Object.keys(existing.payload || {}),
          },
        },
      },
    );
  }
}
const now = Date.now();
```

```
  const docData = {
    userId,
    platform,
    schemaVersion,
    updatedAt,
    baseRemoteUpdatedAt,
    clientFingerprint,
    payload,
    payloadKeys,
    lastWriteAt: now,
  };
  if (existing && existing._id) {
    await col.doc(existing._id).update(docData);
    return { userId, updatedAt, savedAt: now, mode: 'update', sizeBytes: size };
  }
  const addRes = await col.add(docData);
  return {
    userId,
    updatedAt,
    savedAt: now,
    mode: 'insert',
    sizeBytes: size,
    _id: addRes && (addRes.id || addRes._id),
  };
}
module.exports = {
  handlePull,
  handlePush,
};
/**
 * 别捞水果好友榜 openDataContext 入口
 * ----------------------------------------------------------------
 * 运行在独立 JS 上下文（WeChat Mini-Game openDataContext 沙箱）中，
 * 只能访问受限 wx API:
 *   - wx.getSharedCanvas()      获取共享 canvas（主域通过 PIXI Sprite 上屏）
 *   - wx.onMessage(cb)          接收主域发来的渲染参数
 *   - wx.getFriendCloudStorage() 拉好友（含自己）的 KVDataList
 *
 * 主域消息协议（见 src/utils/friendRanking.ts）：
 *   { action: 'render',  tab:'bowl'|'fruit', pixelRatio, width, height, scrollY, selfOpenId, for...
 *
 * UI 风格说明（重要）：
 *   微信不允许主域获取好友明文，绘制必须在子域 canvas API 上完成。
 *   为了让好友榜跟主域的世界榜（src/scenes/LeaderboardScene.ts 中 createRankRow）
 *   看起来完全一致，本文件里的颜色、行高、字号、圆角等全部按主域同名常量 1:1 复刻。
 */
/* eslint-disable */
var CANVAS = null;
var CTX = null;
try {
```

```
  CANVAS = wx.getSharedCanvas();
  CTX = CANVAS.getContext('2d');
} catch (e) {
  // 非小游戏环境（例如开发工具预览），无 sharedCanvas
}
// 跟主域 src/utils/friendRanking.ts:KV_KEY 保持一致
var TAB_META = {
  bowl: { key: 'hotpot_bowl_level', unit: '关' },
  fruit: { key: 'hotpot_fruit_score', unit: '分' },
};
var MEDAL_ASSETS = {
  1: 'assets/images/leaderboard_medal_1.png',
  2: 'assets/images/leaderboard_medal_2.png',
  3: 'assets/images/leaderboard_medal_3.png',
};
var state = {
  tab: 'bowl',
  pixelRatio: 2,
  scrollY: 0,
  selfOpenId: '',
  listCache: {},
  loading: false,
  avatarImgs: {},
  medalImgs: {},
  pendingRender: null,
  inflight: {},
};
var CACHE_TTL_MS = 60 * 1000;
var FONT_FALLBACK = '"PingFang SC","Microsoft YaHei","Helvetica Neue",sans-serif';
// 与主域 LeaderboardScene 内同名常量对齐
var COLOR = {
  CARD_WHITE: '#fdf6ec',
  CARD_STROKE: '#eadbc5',
  ROW_BG: '#fff9ee',
  ROW_STROKE: '#e9d8b9',
  ME_BG: '#ffa743',
  ME_STROKE: '#d6791f',
  TEXT_DARK: '#5a3318',
  TEXT_LIGHT: '#ffffff',
  TEXT_ME_STROKE: '#a14400',
  TEXT_RANK_NUM: '#9b8268',
  MEDAL_GOLD_CORE: '#f7c64a',
  MEDAL_GOLD_EDGE: '#c88517',
  MEDAL_SILVER_CORE: '#d8e2ec',
  MEDAL_SILVER_EDGE: '#8aa3b9',
  MEDAL_BRONZE_CORE: '#e79768',
  MEDAL_BRONZE_EDGE: '#a15a2a',
  MEDAL_RIBBON: '#d94b4b',
  HINT_TEXT: '#8a5a2b',
  HINT_SUB: '#b09060',
```

```
};
var TOP_RANK_ROW_STYLE = {
  1: {
    fill: '#fff3cf',
    stroke: '#efbd48',
    medalCore: '#f8c84f',
    medalEdge: '#c88517',
    ribbon: '#e84848',
  },
  2: {
    fill: '#eaf2ff',
    stroke: '#b5cbe0',
    medalCore: '#d8e2ec',
    medalEdge: '#8aa3b9',
    ribbon: '#5f8fc1',
  },
  3: {
    fill: '#ffeadb',
    stroke: '#e1a078',
    medalCore: '#e79768',
    medalEdge: '#a15a2a',
    ribbon: '#c56a38',
  },
};
function _safeNum(v) {
  var n = parseInt(v, 10);
  return isNaN(n) ? 0 : n;
}
function _isCredentialError(err) {
  var msg = (err && (err.errMsg || err.message)) || '';
  return (err && err.err_code === 40001) || /invalid credential|access_token/i.test(msg);
}
function _parseWxgameData(kvList, tabKey) {
  if (!kvList || !kvList.length) return 0;
  var target = null;
  for (var i = 0; i < kvList.length; i++) {
    if (kvList[i] && kvList[i].key === tabKey) {
      target = kvList[i];
      break;
    }
  }
  if (!target) return 0;
  try {
    var obj = JSON.parse(target.value || '{}');
    if (obj && obj.wxgame && obj.wxgame.score != null) return _safeNum(obj.wxgame.score);
    if (obj && obj.score != null) return _safeNum(obj.score);
    return _safeNum(target.value);
  } catch (_) {
    return _safeNum(target.value);
  }
```

```
}
function _ensureAvatar(url) {
  if (!url) return null;
  var cached = state.avatarImgs[url];
  if (cached) return cached;
  try {
    var img = wx.createImage();
    img.onload = function () {
      state.avatarImgs[url] = img;
      _renderIfReady();
    };
    img.onerror = function () {
      state.avatarImgs[url] = null;
    };
    img.src = url;
    state.avatarImgs[url] = img;
    return img;
  } catch (_) {
    return null;
  }
}
function _ensureMedal(rank) {
  var src = MEDAL_ASSETS[rank];
  if (!src) return null;
  var cached = state.medalImgs[src];
  if (cached) return cached;
  try {
    var img = wx.createImage();
    img.onload = function () {
      state.medalImgs[src] = img;
      _renderIfReady();
    };
    img.onerror = function () {
      state.medalImgs[src] = null;
    };
    img.src = src;
    state.medalImgs[src] = img;
    return img;
  } catch (_) {
    return null;
  }
}
// ---------- 数据拉取 ----------
/**
 * 拉好友 KV（不含定位自己用的 mine KV）。失败信息原样回传，让上层决定提示文案。
 * 拆出来是为了让 _getListForTab 能把它和 _fetchMineCloudStorage 并行起来。
 */
function _fetchFriendCloudStorageOnly(tab, cb) {
  var meta = TAB_META[tab];
  if (!meta) {
```

```
    cb([], null);
    return;
  }
  if (typeof wx === 'undefined' || !wx.getFriendCloudStorage) {
    cb([], { kind: 'unsupported', msg: 'wx.getFriendCloudStorage missing' });
    return;
  }
  try {
    wx.getFriendCloudStorage({
      keyList: [meta.key],
      success: function (res) {
        var rows = (res && res.data) || [];
        var list = [];
        for (var i = 0; i < rows.length; i++) {
          var r = rows[i];
          var value = _parseWxgameData(r.KVDataList, meta.key);
          if (value <= 0) continue;
          list.push({
            openid: r.openid || '',
            nickname: r.nickname || '微信好友',
            avatarUrl: r.avatarUrl || '',
            value: value,
          });
        }
        list.sort(function (a, b) { return b.value - a.value; });
        cb(list, null);
      },
      fail: function (err) {
        var errMsg = (err && (err.errMsg || err.message)) || '';
        var kind = 'empty';
        if (_isCredentialError(err)) kind = 'credential';
        else if (/privacy/i.test(errMsg)) kind = 'privacy';
        else if (/not support|unsupport/i.test(errMsg)) kind = 'unsupported';
        try { console.warn('[openData] getFriendCloudStorage fail', errMsg); } catch (_) {}
        cb([], { kind: kind, msg: errMsg });
      },
    });
  } catch (_) {
    cb([], null);
  }
}
/** 读取自己的微信 KV，用来在好友列表中定位"我"，并绘制底部固定个人信息行 */
function _fetchMineCloudStorage(tab, cb) {
  var meta = TAB_META[tab];
  if (!meta || typeof wx === 'undefined' || !wx.getUserCloudStorage) {
    cb(0);
    return;
  }
  try {
    wx.getUserCloudStorage({
```

```
    keyList: [meta.key],
    success: function (res) {
      cb(_parseWxgameData((res && res.KVDataList) || [], meta.key));
    },
    fail: function (err) {
      if (!_isCredentialError(err)) {
        try { console.warn('[openData] getUserCloudStorage fail', err && (err.errMsg || err)); ...
      }
      cb(0);
    },
  });
 } catch (_) {
  cb(0);
 }
}
/**
 * 并行拉好友 KV + 自己 KV，并尽量"先来先用"地驱动渲染:
 * - 任一端先回 -> 立即触发一次 cb（partial=true，主域已经能开始画骨架/局部内容）
 * - 两端都回 -> 再 cb 一次（final，写入缓存）
 *
 * 这样首屏体感会比"两端都到齐再画"快几百毫秒，
 * 且兜底失败处理跟原先一致（credential / privacy / unsupported 仍能透出）。
 *
 * 通过 state.inflight[tab] 合并并发请求：prefetch + render 同时来时只发一组 wx 请求，
 * 多个 cb 都会在 partial / final 时被依次触发。
 */
function _fetchFriendAndMineParallel(tab, cb) {
 if (!state.inflight) state.inflight = {};
 var running = state.inflight[tab];
 if (running) {
  running.cbs.push(cb);
  return;
 }
 state.loading = true;
 running = {
  friendDone: false,
  mineDone: false,
  friendList: [],
  friendErr: null,
  mineValue: 0,
  firedPartial: false,
  cbs: [cb],
 };
 state.inflight[tab] = running;
 function fireAll(list, err, mineV, isFinal) {
  var snapshot = running.cbs.slice();
  for (var i = 0; i < snapshot.length; i++) {
   try {
    snapshot[i](list, err, mineV, isFinal);
   } catch (e) {
```

```
      try { console.warn('[openData] cb throw', e); } catch (_) {}
    }
  }
}
function tryFire() {
  var allDone = running.friendDone && running.mineDone;
  if (!allDone && (running.friendDone || running.mineDone) && !running.firedPartial) {
    running.firedPartial = true;
    fireAll(running.friendList, running.friendErr, running.mineValue, /*final*/ false);
    return;
  }
  if (allDone) {
    state.loading = false;
    state.listCache[tab] = {
      ts: Date.now(),
      list: running.friendList,
      mineValue: running.mineValue,
      err: running.friendErr || undefined,
    };
    fireAll(running.friendList, running.friendErr, running.mineValue, /*final*/ true);
    delete state.inflight[tab];
  }
}
_fetchFriendCloudStorageOnly(tab, function (list, err) {
  running.friendDone = true;
  running.friendList = list || [];
  running.friendErr = err || null;
  tryFire();
});
_fetchMineCloudStorage(tab, function (v) {
  running.mineDone = true;
  running.mineValue = v || 0;
  tryFire();
});
}
function _getListForTab(tab, forceRefresh, cb) {
  var cached = state.listCache[tab];
  if (!forceRefresh && cached && Date.now() - cached.ts < CACHE_TTL_MS) {
    cb(cached.list, cached.err, cached.mineValue || 0, /*final*/ true);
    return;
  }
  _fetchFriendAndMineParallel(tab, cb);
}
// ---------- 通用绘图 ----------
function _clear() {
  if (!CTX || !CANVAS) return;
  CTX.clearRect(0, 0, CANVAS.width, CANVAS.height);
}
function _renderIfReady() {
  if (state.pendingRender) _render(state.pendingRender);
```

```
}
function _roundRect(x, y, w, h, r) {
  if (w < 2 * r) r = w / 2;
  if (h < 2 * r) r = h / 2;
  CTX.beginPath();
  CTX.moveTo(x + r, y);
  CTX.arcTo(x + w, y, x + w, y + h, r);
  CTX.arcTo(x + w, y + h, x, y + h, r);
  CTX.arcTo(x, y + h, x, y, r);
  CTX.arcTo(x, y, x + w, y, r);
  CTX.closePath();
}
/** 仿主域 PIXI.Text 的描边文字效果：先描边再填充 */
function _drawStrokedText(text, x, y, fontSize, weight, fill, stroke, strokeWidth) {
  CTX.font = weight + ' ' + fontSize + 'px ' + FONT_FALLBACK;
  if (strokeWidth > 0 && stroke) {
    CTX.lineWidth = strokeWidth;
    CTX.strokeStyle = stroke;
    CTX.lineJoin = 'round';
    CTX.strokeText(text, x, y);
  }
  CTX.fillStyle = fill;
  CTX.fillText(text, x, y);
}
function _drawHint(text, sub) {
  if (!CTX) return;
  var w = CANVAS.width;
  var h = CANVAS.height;
  var S = state.pixelRatio || 2;
  _clear();
  CTX.save();
  CTX.textAlign = 'center';
  CTX.textBaseline = 'middle';
  _drawStrokedText(
    text || '暂无好友数据',
    w / 2,
    h / 2 - 14 * S,
    26 * S,
    '900',
    COLOR.TEXT_DARK,
    COLOR.TEXT_LIGHT,
    3 * S
  );
  if (sub) {
    CTX.fillStyle = COLOR.HINT_SUB;
    CTX.font = '700 ' + (20 * S) + 'px ' + FONT_FALLBACK;
    CTX.fillText(sub, w / 2, h / 2 + 24 * S);
  }
  CTX.restore();
}
```

```
// ---------- 主行渲染：对齐主域 createRankRow / createRankBadge / createValueText ----------
/** 圆形头像：白色底盘 + 描边，未加载时只剩底盘（自动 emoji 兜底放主域，子域只画图） */
function _drawAvatar(cx, cy, S, url, isSelf) {
  var r = 30 * S;
  // 白色底盘
  CTX.beginPath();
  CTX.arc(cx, cy, r, 0, Math.PI * 2);
  CTX.fillStyle = '#ffffff';
  CTX.fill();
  CTX.lineWidth = 3 * S;
  CTX.strokeStyle = isSelf ? COLOR.TEXT_ME_STROKE : COLOR.ROW_STROKE;
  CTX.stroke();
  var img = _ensureAvatar(url);
  if (img && img.width > 0) {
    CTX.save();
    CTX.beginPath();
    CTX.arc(cx, cy, r - 3 * S, 0, Math.PI * 2);
    CTX.clip();
    try {
      CTX.drawImage(img, cx - r, cy - r, r * 2, r * 2);
    } catch (_) {}
    CTX.restore();
  } else {
    // 加载未完成时给一个透明米色填充，避免行间空白
    CTX.save();
    CTX.beginPath();
    CTX.arc(cx, cy, r - 3 * S, 0, Math.PI * 2);
    CTX.fillStyle = 'rgba(245,185,74,0.2)';
    CTX.fill();
    CTX.restore();
  }
}
/** 名次徽章：前 3 名画"丝带 + 圆盘 + 数字"；4+ 名显示纯数字 */
function _drawBadge(cx, cy, S, rank, isSelf) {
  if (rank <= 3) {
    var palette = TOP_RANK_ROW_STYLE[rank];
    var medalImg = _ensureMedal(rank);
    if (medalImg && medalImg.width > 0) {
      try {
        CTX.drawImage(medalImg, cx - 41 * S, cy - 41 * S, 82 * S, 82 * S);
        return;
      } catch (_) {}
    }
    // 丝带（两条三角形），跟主域 createRankBadge 内坐标比例对齐
    CTX.fillStyle = palette.ribbon;
    CTX.beginPath();
    CTX.moveTo(cx - 14 * S, cy - 8 * S);
    CTX.lineTo(cx - 2 * S, cy - 8 * S);
    CTX.lineTo(cx - 2 * S, cy + 28 * S);
    CTX.lineTo(cx - 9 * S, cy + 22 * S);
```

```
    CTX.lineTo(cx - 16 * S, cy + 28 * S);
    CTX.closePath();
    CTX.fill();
    CTX.beginPath();
    CTX.moveTo(cx + 14 * S, cy - 8 * S);
    CTX.lineTo(cx + 2 * S, cy - 8 * S);
    CTX.lineTo(cx + 2 * S, cy + 28 * S);
    CTX.lineTo(cx + 9 * S, cy + 22 * S);
    CTX.lineTo(cx + 16 * S, cy + 28 * S);
    CTX.closePath();
    CTX.fill();
    // 圆盘：外圈 + 内圈
    CTX.beginPath();
    CTX.arc(cx, cy, 30 * S, 0, Math.PI * 2);
    CTX.fillStyle = palette.medalEdge;
    CTX.fill();
    CTX.beginPath();
    CTX.arc(cx, cy, 25 * S, 0, Math.PI * 2);
    CTX.fillStyle = palette.medalCore;
    CTX.fill();
    CTX.beginPath();
    CTX.arc(cx - 8 * S, cy - 9 * S, 8 * S, 0, Math.PI * 2);
    CTX.fillStyle = 'rgba(255,255,255,0.35)';
    CTX.fill();
    // 名次数字
    CTX.textAlign = 'center';
    CTX.textBaseline = 'middle';
    _drawStrokedText(
      String(rank),
      cx,
      cy,
      28 * S,
      '900',
      COLOR.TEXT_DARK,
      COLOR.TEXT_LIGHT,
      3 * S
    );
    return;
  }
  // 4+ 名：纯数字（自己行白字 + 深橙描边，其他玩家米色字 + 白描边）
  var display = isSelf && rank > 99 ? '99+' : String(rank);
  CTX.textAlign = 'center';
  CTX.textBaseline = 'middle';
  _drawStrokedText(
    display,
    cx,
    cy,
    (display === '99+' ? 26 : 32) * S,
    '900',
    COLOR.TEXT_RANK_NUM,
```

```
    COLOR.TEXT_LIGHT,
    2 * S
  );
}
/** 单行：跟世界榜风格完全一致 */
function _drawRow(rowX, rowY, rowW, rowH, S, meta, item, rank, isSelf) {
  var rowR = 16 * S;
  var topStyle = TOP_RANK_ROW_STYLE[rank];
  // 行底色 + 描边
  if (topStyle) {
    _roundRect(rowX, rowY, rowW, rowH, rowR);
    CTX.fillStyle = topStyle.fill;
    CTX.fill();
    CTX.lineWidth = 3 * S;
    CTX.strokeStyle = topStyle.stroke;
    CTX.stroke();
  } else if (isSelf) {
    _roundRect(rowX, rowY, rowW, rowH, rowR);
    CTX.fillStyle = '#ffedb0';
    CTX.fill();
    CTX.lineWidth = 3 * S;
    CTX.strokeStyle = '#efbd48';
    CTX.stroke();
  } else {
    CTX.beginPath();
    CTX.moveTo(rowX + 18 * S, rowY + rowH);
    CTX.lineTo(rowX + rowW - 18 * S, rowY + rowH);
    CTX.lineWidth = 1 * S;
    CTX.strokeStyle = 'rgba(233,216,185,0.75)';
    CTX.stroke();
  }
  // 名次徽章 / 圆形头像 / 名字 / 分数 的水平偏移：
  // 主域 createRankRow 内左基 = -w/2 + offset；下面把它转成行左边距 + offset
  var badgeCx = rowX + 36 * S;
  var badgeCy = rowY + rowH / 2 + (topStyle ? -2 * S : 0);
  _drawBadge(badgeCx, badgeCy, S, rank, isSelf);
  var avatarCx = rowX + 112 * S;
  var avatarCy = rowY + rowH / 2;
  _drawAvatar(avatarCx, avatarCy, S, item.avatarUrl, isSelf);
  var nameX = rowX + 170 * S;
  var nameY = rowY + rowH / 2;
  CTX.textAlign = 'left';
  CTX.textBaseline = 'middle';
  var nick = (item.nickname || '微信好友').substring(0, 8);
  _drawStrokedText(
    nick,
    nameX,
    nameY,
    (topStyle ? 28 : 25) * S,
    '900',
```

```
    COLOR.TEXT_DARK,
    COLOR.TEXT_LIGHT,
    (topStyle ? 2 : 1) * S
   );
   var unit = meta.unit || '';
   var valText = item.value + unit;
   CTX.textAlign = 'right';
   CTX.textBaseline = 'middle';
   _drawStrokedText(
    valText,
    rowX + rowW - 32 * S,
    rowY + rowH / 2,
    26 * S,
    '900',
    '#d25a36',
    COLOR.TEXT_LIGHT,
    2 * S
   );
  }
  function _resolveMineRow(list, mineValue) {
   if (!list || !list.length) {
    return {
     item: { nickname: '自己', avatarUrl: '', value: mineValue || 0 },
     rank: mineValue > 0 ? 1 : 100,
    };
   }
   if (state.selfOpenId) {
    for (var i = 0; i < list.length; i++) {
     if (list[i].openid === state.selfOpenId) {
      return { item: list[i], rank: i + 1 };
     }
    }
   }
   // 主域拿不到好友明文 openid 时，用自己的 KV 分数回定位。
   // 极少数同分会命中第一个同分好友，但底部仍能保持世界榜同款“个人信息”布局。
   if (mineValue > 0) {
    for (var j = 0; j < list.length; j++) {
     if (list[j].value === mineValue) {
      return { item: list[j], rank: j + 1 };
     }
    }
    var rank = 1;
    for (var k = 0; k < list.length; k++) {
     if (list[k].value > mineValue) rank += 1;
    }
    return {
     item: { nickname: '自己', avatarUrl: '', value: mineValue },
     rank: rank,
    };
   }
```

```
  return {
    item: { nickname: '自己', avatarUrl: '', value: 0 },
    rank: 100,
  };
}
function _drawBoard(list, mineValue) {
  var S = state.pixelRatio || 2;
  var w = CANVAS.width;
  var h = CANVAS.height;
  // 与主域 drawList + drawMineRow 完全一致：上方列表预留底部个人信息行。
  var rowH = 84 * S;
  var gap = 10 * S;
  var startY = 0;
  var mineGap = 26 * S;
  var listH = Math.max(0, h - rowH - mineGap);
  var rawScrollY = state.scrollY || 0;
  var totalH = list.length * (rowH + gap);
  var minScrollY = Math.min(0, listH - startY - totalH);
  var scrollY = Math.max(minScrollY, Math.min(0, rawScrollY));
  var meta = TAB_META[state.tab] || TAB_META.bowl;
  var mine = _resolveMineRow(list, mineValue);
  _clear();
  CTX.save();
  CTX.beginPath();
  CTX.rect(0, 0, w, listH);
  CTX.clip();
  for (var i = 0; i < list.length; i++) {
    var item = list[i];
    var ry = startY + i * (rowH + gap) + scrollY;
    if (ry + rowH < 0 || ry > listH) continue;
    var isSelf = (state.selfOpenId && item.openid === state.selfOpenId)
      || (mineValue > 0 && item.value === mineValue && i + 1 === mine.rank);
    _drawRow(0, ry, w, rowH, S, meta, item, i + 1, isSelf);
  }
  CTX.restore();
  // 底部固定个人信息行：位置、尺寸、橙色高亮都按世界榜 drawMineRow 对齐。
  _drawRow(0, h - rowH, w, rowH, S, meta, mine.item, mine.rank, true);
}
/**
 * 把"拿到的数据状态"翻成一次具体的绘制。
 * 拆开是因为并行拉取时会被回调两次（partial + final），两次都要走同一套画法。
 */
function _paintResult(list, err, mineValue) {
  if (err && err.kind === 'privacy') {
    _drawHint('好友榜暂不可用', '请检查小程序后台隐私协议配置');
    return;
  }
  if (err && err.kind === 'credential') {
    _drawHint('好友榜暂不可用', '微信凭证异常，重进小程序后再试');
    return;
```

```
  }
  if (err && err.kind === 'unsupported') {
    _drawHint('好友榜暂不可用', '请升级微信到最新版本重试');
    return;
  }
  if (!list || !list.length) {
    if (mineValue > 0) {
      _drawBoard([], mineValue);
    } else {
      _drawHint('暂无好友上榜', '邀请微信好友一起来玩即可上榜');
    }
    return;
  }
  _drawBoard(list, mineValue || 0);
}
function _render(msg) {
  state.pendingRender = msg;
  if (!CTX || !CANVAS) {
    try { console.warn('[openData] render skipped: no CTX/CANVAS'); } catch (_) {}
    return;
  }
  state.tab = msg.tab || state.tab;
  state.pixelRatio = msg.pixelRatio || state.pixelRatio;
  state.scrollY = msg.scrollY != null ? msg.scrollY : state.scrollY;
  state.selfOpenId = msg.selfOpenId || state.selfOpenId;
  // 新基础库下 sharedCanvas 的 width/height 在 openDataContext 内只读，
  // 必须由主域写入（见 src/utils/friendRanking.ts:ensureSharedCanvasSize）。
  // 进入这一帧能拿到啥就先画啥，绝不留白屏：
  //  - 有缓存 -> 直接用缓存数据画一份（不管 force，避免 force 重拉时白屏空等）；
  //  - 无缓存 -> 画一个 loading 骨架占位。
  // 后续 _getListForTab 拉到新数据时会再覆盖一次（partial / final 两次都会刷）。
  var cached = state.listCache[state.tab];
  var hasFresh = cached && Date.now() - cached.ts < CACHE_TTL_MS;
  try {
    console.log(
      '[openData] render tab=' + state.tab + ' force=' + !!msg.force
        + ' hasFreshCache=' + !!hasFresh
        + ' canvas=' + (CANVAS.width + 'x' + CANVAS.height)
    );
  } catch (_) {}
  if (hasFresh) {
    _paintResult(cached.list, cached.err, cached.mineValue || 0);
  } else if (!msg.silent) {
    _drawHint('正在加载好友榜...', '正在向微信请求好友数据');
  }
  _getListForTab(state.tab, !!msg.force, function (list, err, mineValue, isFinal) {
    if (!isFinal && (!list || !list.length) && !err && !(mineValue > 0)) {
      return;
    }
    try {
```

```
      console.log(
        '[openData] paint tab=' + state.tab
          + ' isFinal=' + isFinal
          + ' listLen=' + ((list && list.length) || 0)
          + ' mineValue=' + (mineValue || 0)
          + ' err=' + (err ? err.kind : 'none')
      );
    } catch (_) {}
    _paintResult(list, err, mineValue);
  });
}
/**
 * 静默预拉：把当前 tab 的好友/自己 KV 拉到子域 60s 缓存里，但不画任何东西。
 *
 * 主域 LeaderboardScene 在还停留在世界榜时调用，等用户切到好友榜时直接命中缓存，
 * 跳过整段 wx 网络往返。如果用户从不切到好友榜，多出来的两次 KV 网络也只是一次性的，
 * 不会持续刷。
 */
function _prefetch(tab) {
  if (!tab) tab = state.tab;
  state.tab = tab || state.tab;
  var cached = state.listCache[state.tab];
  if (cached && Date.now() - cached.ts < CACHE_TTL_MS) return;
  _fetchFriendAndMineParallel(state.tab, function () {
    // 结果已经写进 state.listCache 由 _fetchFriendAndMineParallel 负责，无需绘制
  });
}
// ---------- 消息入口 ----------
try {
  console.log(
    '[openData] subdomain boot, hasCANVAS=' + !!CANVAS + ' hasCTX=' + !!CTX
      + ' wx=' + (typeof wx !== 'undefined')
      + ' wxOnMessage=' + (typeof wx !== 'undefined' && typeof wx.onMessage === 'function')
  );
} catch (_) {}
if (typeof wx !== 'undefined' && wx.onMessage) {
  wx.onMessage(function (data) {
    try { console.log('[openData] onMessage action=' + (data && data.action)); } catch (_) {}
    if (!data || !data.action) return;
    if (data.action === 'render' || data.action === 'refresh') {
      _render(data);
    } else if (data.action === 'prefetch') {
      _prefetch(data.tab);
    } else if (data.action === 'invalidate') {
      // 主域刚 setUserCloudStorage 完，下次 render 时强制重新拉。
      // 在飞的请求已经发出了，让它完成并写入新缓存即可；这里只清旧缓存。
      state.listCache = {};
    }
  });
}
```